AI-Powered Full Stack Developer Program

STUDENT NOTES

Week 2 · Day 1

How the Web Works + HTML Core

Theory notes 01–11: read them before homework, keep them for interviews

Momo House · TSI Canteen · Aarav · Sunita · Didi

Stage 1, AI as Tutor: AI may explain; you type every line.

Tech Skills Institute (TSI) Nepal · Kathmandu

Week 2 · Day 1 Notes: How the Web Works + HTML Core

File: notes/README.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2, Day 1 · Layer: Frontend

The One Rule: You may only ship code you can explain, line by line.

These notes are your take-home copy of today's class. Read one note before each homework task. Keep the cheatsheet (note 11) open while you code.

How to use these notes

1. Open the `W2D1` folder in VS Code (File → Open Folder).
2. When a note shows `File: index.html`, make a **new file** with that name in a practice folder, type the code (do not paste it), save, and right-click → **Open with Live Server**.
3. Before you save, read the **Predict:** line. Guess what you will see. Then check.
4. Each note ends with **Check yourself** questions. There are no answers on the page on purpose. Find the answer by running code, or ask in class.
5. Images: the snippets use files like `images/momo.svg`. Copy the `images` folder from any classwork folder (for example `classwork/D1-09-broken-paths/images/`) into your practice folder, next to your HTML file.

The notes

#	Note	Class block	Practise with
01	How the web works	B1	D1-01
02	VS Code, Emmet and Live Server	B0 + B2	D1-00, D1-02
03	The HTML skeleton and anatomy	B3	D1-03, D1-04
04	Text tags and entities	B4	D1-05, D1-06, D1-FF1
05	Lists	B5	D1-07, D1-08
06	Links and paths	B6	D1-09, D1-FF2, D1-FF6
07	Images	B6	D1-10, D1-FF3
08	Tables	B7	D1-11, D1-12
09	Forms	B8	D1-13, D1-14
10	Debugging and validating HTML	B3 spell-checker + B9 + B10	D1-04, D1-15, D1-16, D1-FF4, D1-FF5
11	Day 1 cheatsheet	all	everything

The bug tags you will see in every note

Every planted bug in class has a tag. It tells you **how** you can find it.

Tag	Meaning	Tool that shows it
V	You can **see** it on the page	your eyes
D(HH)	HTMLHint underlines it in VS Code / Problems panel (Ctrl+Shift+M)	HTMLHint with the class `.htmlhintrc`
D(F12)	Only the Elements panel shows what the browser actually built	Chrome DevTools
D(URL)	Submit the form (`method="get"`) and read the address bar	the URL bar
D(LH)	Lighthouse → Accessibility report (works offline)	Chrome DevTools, Day 2 only
D(W3C)	Only the W3C validator reports it	validator.w3.org (Wi-Fi bonus)
H	Hidden on Windows, breaks on a real Linux server (file-name case)	explained by the instructor
R	**Rule check**: no tool fires; you must read the code against the rule (e.g. "one h1", "no heading jumps")	your brain + the notes

The story you will meet

- **Momo House**: Didi's momo restaurant in Kathmandu. Its website broke the night before opening. You repair it.
- **TSI Canteen**: the canteen downstairs, with a price board and a weekly news sheet.
- **Aarav** and **Sunita Gurung**: two students in your batch. Aarav makes many beginner mistakes, so you will fix his pages.
- **Didi**: owner of the canteen and of Momo House.

AI rule for this week

Stage 1: AI as Tutor. AI may **explain** a concept or an error. You type every line yourself. Try first, then ask.

Remember the One Rule: you may only ship code you can explain, line by line. Start with note 01: How the web works.

01 · How the Web Works: Student Notes

File: notes/01-how-the-web-works.md

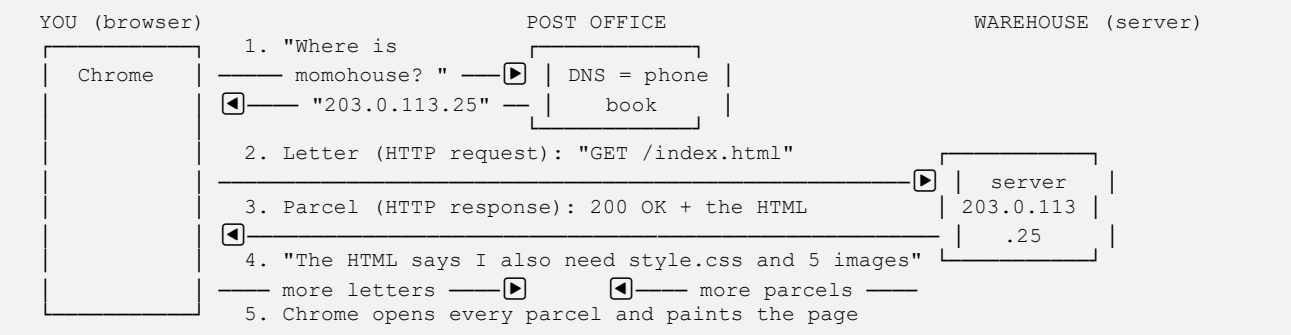
TSI Nepal · AI-Powered Full Stack Developer Program · Week 2, Day 1 · Block B1 · Layer: Frontend (and the network under it)

Today in one sentence: The internet is not magic. It is computers mailing text to each other.

The picture to keep in your head

The Internet is a postal system.

- The **browser** = you, writing a letter with a request ("please send me the Momo House home page").
- The **HTTP request** = the letter.
- The **IP address** = a precise home address (like `House 14, Dillibazar, Kathmandu`, but in numbers).
- **DNS** = the phone book. You know the name (`momohouse.com.np`); the phone book gives you the address (`203.0.113.25`).
- The **server** = a warehouse that mails back the goods.
- The **response** = the parcel (HTML, CSS, images).



Postal system	The web
You write a letter	The browser makes an **HTTP request**
The phone book	**DNS** (Domain Name System)
A home address	An **IP address**
The warehouse	A **server** (a computer that is always on and answers requests)
The parcel that comes back	The **HTTP response** (status code + the file)
One order, many parcels	One page = many files (HTML, CSS, every image)

A website is a restaurant (recap from Week 1)

Restaurant	Web	Today?
Ingredients and the menu structure	**HTML**: what is on the page	Day 1 (today)
Plating and decor	**CSS**: how it looks	Day 2
The waiter who reacts to you	**JavaScript**: behaviour	Later (session 4)

Words you now own

Term	What it means (plain English)	Example
Browser	The program that asks for pages and paints them	Chrome, Edge, Firefox
Server	A computer that stores files and mails them to whoever asks	The computer that hosts `momohouse.com.np`

Term	What it means (plain English)	Example
Client	The side that asks. Your browser is the client	Chrome on your laptop
URL	The full address of one file on the web	`https://momohouse.com.np/menu.html`
Domain name	The human name of a server	`momohouse.com.np`
IP address	The number address of a computer	`203.0.113.25`, `127.0.0.1` (your own computer)
DNS	The phone book that turns a domain name into an IP address	`momohouse.com.np` → `203.0.113.25`
HTTP / HTTPS	The language of the letters. HTTPS = the same, in a sealed envelope (encrypted)	`https://` at the start of a URL
Request	The letter: "please send me this file"	`GET /index.html`
Response	The parcel: a status code + the file	`200 OK` + the HTML
Status code	A number on the parcel that says how it went	`200` OK, `404` Not Found, `500` server error
`ping`	A command that sends a tiny test letter and times the reply	`ping localhost`
View Source	Shows the raw HTML text the server sent	Right-click → View Page Source (Ctrl+U)
Network tab	F12 panel that lists every parcel the page needed	F12 → Network → refresh

How it works

1. Reading a URL

```
https://momohouse.com.np/pages/menu.html#specials
└──┘   └──────┬──────┘└──────┬──────┘└──┬──┘
scheme       domain          path       fragment
(how)     (which server)  (which file)  (which spot on the page)
```

- The **scheme** says which language the letters use: `https` (sealed) or `http` (open postcard).
- The **domain** is looked up in DNS to get the IP address.
- The **path** is the folder and file on the server. Same idea as folders on your laptop.
- The **fragment** (`#specials`) never goes to the server. The browser uses it to scroll to `id="specials"`.

2. DNS: the phone book

Computers only understand number addresses. People remember names. DNS is the service that answers "what is the number for this name?". Your laptop asks DNS once, then remembers the answer for a while (this memory is called a **cache**).

3. Request and response

The browser sends a short text letter. The server sends back a parcel with a status line, some labels (headers) and the file. A simplified real request looks like this:

```
GET /index.html HTTP/1.1
Host: momohouse.com.np
```

And the response:

```
HTTP/1.1 200 OK
Content-Type: text/html

<!DOCTYPE html>
<html lang="en"> ...
```

It really is just text going back and forth. That is why we say "computers mailing text to each other".

4. One page, many parcels

The first parcel is the HTML. Chrome reads it and finds more things it needs: a `<link>` to `style.css`, and an `<img>` for every picture. **Each one is a new letter and a new parcel.** A page with 1 HTML file, 1 CSS file and 5 images needs **7 parcels**. You will count them yourself in D1-01. Live Server adds one more row called `ws`. It is not a parcel. Don't count it.

5. Status codes you will see this week

Code	Meaning	When you will meet it
200	OK, here is your file	Every file that loads
304	Not Modified: "you already have it, use your copy"	Refresh a page twice (Chrome may show "(memory cache)" instead: same idea)
404	Not Found: no file at that path	A broken link or a wrong image path (D1-09)
500	The server itself broke	Later, when you write servers

6. Live Server is a tiny warehouse on your own laptop

When you click **Open with Live Server**, VS Code starts a small server on your own computer. The address looks like `http://127.0.0.1:5500/index.html`.

- `127.0.0.1` (also called `localhost`) means "this computer". The letter never leaves your laptop.
- `5500` is the **port**: which door of the warehouse to knock on.

So even with no Wi-Fi, you can watch real requests and responses in class.

7. Browser = translator

The server sends plain text. The browser translates that text into a page you can see: headings, pictures, colours. **View Source** shows the text before translation. The **Elements** panel (F12) shows the page after translation. (In note 10 you will see that these two can be different.)

Code you built today

Step 1: the parcel page

Make a folder `parcel-page` with an `images` folder inside. Copy in `logo-momo-house.svg`, `momo.svg`, `momo-plate.svg`, `thamel.svg` and `swayambhu.svg`.

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House · Parcel Page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <img src="images/logo-momo-house.svg" alt="Momo House logo" width="160">
  <h1>Welcome to Momo House</h1>
  <p>Every picture on this page is a separate parcel.</p>
  <img src="images/momo.svg" alt="A single steamed momo" width="150">
  <img src="images/momo-plate.svg" alt="A plate of ten momos" width="150">
  <img src="images/thamel.svg" alt="A street in Thamel" width="150">
  <img src="images/swayambhu.svg" alt="Swayambhu stupa on its hill" width="150">
</body>
</html>
```

File: `style.css`

```
body {
  font-family: Arial, sans-serif;
  background-color: #fdf6ec;
  color: #14213d;
}

h1 {
  color: #990011;
}
```

- `<link rel="stylesheet" href="style.css">` is a request for **one more parcel**: the CSS file.
- Each `<img src="...">` is **another parcel**.

Predict: how many rows will the Network tab show after you refresh? Write your number down first.

Step 2: watch the parcels arrive

1. Open `index.html` with Live Server.
2. Press **F12** → click the **Network** tab.
3. Press **F5** (refresh). Rows appear: one per parcel. Live Server adds one more row called `ws`. It is not a parcel. Don't count it.
4. Look at these columns:
   - **Name**: which file.
   - **Status**: `200` (or `304` / "from cache" the second time).
   - **Type**: `document` (the HTML), `stylesheet`, `svg+xml`.
   - **Size** and **Time**: how big, how long.
5. Rename one image in the HTML to a wrong name, save and refresh. **Predict:** what status will that row show?

Step 3: View Source

Right-click the page → **View Page Source** (or Ctrl+U). You see the exact text the server mailed. Find the `<h1>`. Nothing is hidden: every website's HTML is public. That is how many developers learnt.

Step 4: `ping`

Open a terminal (in VS Code: Ctrl+\`) and type:

```
ping localhost
```

Each line is one tiny letter sent and its reply received. `time<1ms` means the round trip took under a thousandth of a second, because the letter never left your laptop.

With Wi-Fi, try:

```
ping google.com
```

- The first line shows the IP address DNS found for `google.com`. That is the phone book at work.
- The time is now bigger (for example 30–80 ms): the letter travels across Nepal and out of the country and back.
- On Windows `ping` stops after 4 letters. On Mac/Linux, press Ctrl+C to stop.

Quick reference

You want to...	Do this
See every file a page loaded	F12 → Network → F5
See the raw HTML the server sent	Ctrl+U (View Page Source)
See what the browser built	F12 → Elements
Test if a computer answers	`ping localhost` / `ping google.com`
Start your own little server	Right-click the HTML file → Open with Live Server

You want to...	Do this
Check the address of your page	Look at the URL bar: `http://127.0.0.1:5500/...`

Mistakes everyone makes

You see...	Because...	Fix
The Network tab is empty	You opened it **after** the page loaded	Keep F12 open and press F5
An image row shows **404** in red	The path in `src` does not match the real file (see D1-09 bug 1)	Check the folder and file name letter by letter
The URL starts with `file:///C:/...`	You double-clicked the file instead of using Live Server	That is fine as a fallback; press F5 after each save. The Network tab still works
`ping google.com` says "could not find host"	No Wi-Fi, so DNS (the phone book) cannot answer	Use `ping localhost`; the offline path still works
You think the server "sends a page"	It sends **text files**. The browser builds the page	Remember: browser = translator
Your count of parcels is 2 (HTML + CSS)	You forgot that every image is its own parcel	Count every `<img>` and every `<link>`
Your count is 8, not 7	You counted the `ws` row. Live Server adds it to reload the page when you save	`ws` is not a parcel. Don't count it

Check yourself

1. In the postal system, what are DNS, the IP address and the server?
2. Put these in order: Chrome paints the page · DNS answers · the server sends the HTML · Chrome asks for `style.css`.
3. A page has 1 HTML file, 2 CSS files and 8 images. How many parcels? Why?
4. What does status `404` tell you? Where have you seen it today?
5. What is `127.0.0.1`? Why does `ping localhost` work with no Wi-Fi?
6. What is the difference between View Source and the Elements panel?
7. Which part of `https://momohouse.com.np/menu.html#momo` never travels to the server?
8. Explain to a friend who has never coded what happens when they type a web address and press Enter. Use the postal system. Five sentences maximum.

Try it

- **D1-01 See the letters arrive**: `ping`, the Network tab, View Source. Write your parcel count in `my-findings.md`.
- At home (Wi-Fi): open any big site (for example a news site), open the Network tab and refresh. How many parcels? Write the number in your notebook and bring it to class.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. Example prompt:

> "I am a beginner. Using the postal-system idea, explain what DNS does when I type a website name. Do not give me any code."

Remember the One Rule: you may only ship code you can explain, line by line. Next: 02 · VS Code, Emmet and Live Server.

02 · VS Code, Emmet and Live Server: Student Notes

File: notes/02-vscode-emmet-live-server.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2, Day 1 · Blocks B0 + B2 · Layer: Frontend (your tools)

Today in one sentence: A good cook keeps the right tools within reach. A good developer does too.

The picture to keep in your head

A professional kitchen: labeled tools within reach; **"taste as you cook" = Live Server**. Emmet = **speed-dial for tags**.

Kitchen	VS Code
Check the stove works before cooking	D1-00 setup check: open the folder, start Live Server
Knives and spoons within reach	Keyboard shortcuts
Taste as you cook	**Live Server**: save, and the page reloads by itself
Speed-dial on a phone: press 2, it calls Didi	**Emmet**: type `ul>li*3`, press Tab, get a whole list
A friend who spots salt instead of sugar	**HTMLHint**: red/yellow squiggles under mistakes (note 10)

Words you now own

Term	What it means (plain English)	Example
Workspace folder	The one folder you open in VS Code. All paths start here	File → Open Folder → `W2D1`
Explorer	The file list on the left (Ctrl+Shift+E)	Shows `classwork/`, `notes/`
Extension	An add-on that gives VS Code a new skill	Live Server, HTMLHint
Live Server	An extension that runs a tiny server and reloads the page when you save	`http://127.0.0.1:5500/index.html`
Emmet	A shorthand built into VS Code: a short code expands into full HTML	`!` + Tab → a full skeleton
Abbreviation	The short code you type for Emmet	`nav>a*3`
Command Palette	A search box for every VS Code command (Ctrl+Shift+P)	"Format Document"
Linked editing	Change an opening tag and the closing tag changes too	`div` → `section` in both places
Problems panel	The list of every error HTMLHint found (Ctrl+Shift+M)	"Tag must be paired"
Autosave	Saves for you. **Off** in this class, so you learn to press Ctrl+S	`"files.autoSave": "off"`

How it works

1. Always open the folder, not the file

Open **File → Open Folder** and choose `W2D1`. Do not double-click single files into VS Code.

- When you open a folder, VS Code reads the class settings in `W2D1/.vscode/settings.json` (2-space tabs, linked editing) and the HTMLHint rules in `W2D1/.htmlhintrc`.
- Live Server uses the open folder as the root of its little warehouse.
- If you open only one file, links and images to other files may break and HTMLHint may stay quiet.

2. Live Server: taste as you cook

1. Right-click an `.html` file in the Explorer → **Open with Live Server** (or click **Go Live** at the bottom right).
2. Chrome opens `http://127.0.0.1:5500/...`.
3. Change something, press **Ctrl+S**. The page reloads by itself.

No Live Server yet? Double-click the HTML file in Windows File Explorer. It opens in Chrome as `file:///C:/...`. After each save, press **F5** in Chrome. Install Live Server at the first break.

3. Emmet: speed-dial for tags

Type an abbreviation in an `.html` file and press **Tab** (or Enter when the suggestion box shows).

Symbol	Means	Abbreviation	Gives you
`!`	a full HTML skeleton	`!`	DOCTYPE, html, head, body
`>`	child (put inside)	`nav>a`	`<nav><a href=""></a></nav>`
`+`	sibling (put next to)	`h1+p`	`<h1></h1><p></p>`
`*`	repeat	`li*3`	three `<li></li>`
`.`	class	`.card`	`<div class="card"></div>`
`#`	id	`section#menu`	`<section id="menu"></section>`
`$`	a number that counts up	`li.item$*3`	`item1`, `item2`, `item3`
`{}`	text inside	`a{Home}`	`<a href="">Home</a>`
`lorem`	dummy text	`lorem20`	20 words of fake Latin text

Combine them:

```
ul>li*5             → a list with 5 items
nav>a*3             → a nav with 3 links
table>tr*3>td*2     → a table: 3 rows, 2 cells in each
.card>h2+p          → a div.card with an h2 and a p inside
ul>li.item$*3       → <li class="item1">, <li class="item2">, <li class="item3">
```

Important: Emmet's `!` writes `<html lang="en">` and `<title>Document</title>`. Always change `Document` to a real title. "Document" is not a title a visitor can use.

4. Shortcuts within reach

Shortcut	What it does	When to use it
Ctrl+S	Save	After every change. Live Server reloads only when you save
Alt+Shift+↓	Copy the current line down	Repeat a `<li>` or a table row
Alt+↑ / Alt+↓	Move the line up / down	Put a tag in the right place
Ctrl+/	Comment / uncomment the line	Hide a line while you test
Shift+Alt+F	Format document (fix indentation)	Messy code; shows nesting mistakes
Ctrl+D	Select the next match of the same word	Rename three `td` at once
Ctrl+Shift+M	Problems panel	See every HTMLHint error
Ctrl+\`	Open the terminal	`ping localhost`
Ctrl+P	Quick open a file by name	Jump to `menu.html`
Ctrl+Z	Undo	Always

Linked editing: put the cursor in `div` of `<div>` and type `section`. The closing `</div>` becomes `</section>` at the same time. Our class settings switch it on (`"editor.linkedEditing": true`).

Code you built today

Step 1: setup check (D1-00)

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Setup check · Aarav</title>
</head>
<body>
  <h1>Hello, I am Aarav</h1>
  <p>My VS Code and Live Server work.</p>
</body>
</html>
```

- Change `Aarav` to your own name. Press Ctrl+S.
- **Predict:** do you need to press F5 in Chrome? (With Live Server: no. With the double-click fallback: yes.)
- Press **F12**. You have opened DevTools. You will use it all day.

Step 2: the Emmet skeleton

In a new empty file `emmet.html`, type `!` and press Tab. You get this:

File: `emmet.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Emmet drill · Momo House</title>
</head>
<body>
  <h1>Momo House</h1>
  <p>Fresh momo every day in Kathmandu.</p>
</body>
</html>
```

- The body came from `h1+p` + Tab, then typing the text.
- `Document` was replaced by a real title.

Step 3: Emmet structures

Each block below was made by one abbreviation (shown above the block), then filled with text.

File: `structures.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Emmet structures · Momo House</title>
</head>
<body>
  <h1>Emmet structures</h1>

  <!-- nav>a*3 -->
  <nav>
    <a href="#menu">Menu</a>
    <a href="#prices">Prices</a>
    <a href="#contact">Contact</a>
  </nav>

  <!-- ul>li*5 -->
  <h2 id="menu">Menu</h2>
  <ul>
    <li>Chicken momo</li>
    <li>Veg momo</li>
    <li>Buff momo</li>
    <li>Paneer momo</li>
    <li>Chocolate momo</li>
  </ul>
```

```
  <!-- table>tr*3>td*2 -->
  <h2 id="prices">Prices</h2>
  <table>
    <tr>
      <td>Steam</td>
      <td>Rs. 150</td>
    </tr>
    <tr>
      <td>Fried</td>
      <td>Rs. 180</td>
    </tr>
    <tr>
      <td>Jhol</td>
      <td>Rs. 200</td>
    </tr>
  </table>

  <!-- .card>h2+p -->
  <div class="card">
    <h2 id="contact">Contact</h2>
    <p>Call Didi at the counter.</p>
  </div>

  <!-- ul>li.item$*3 -->
  <ul>
    <li class="item1">First</li>
    <li class="item2">Second</li>
    <li class="item3">Third</li>
  </ul>
</body>
</html>
```

- `<!-- ... -->` is a comment: a note for people. The browser ignores it (note 03).
- **Predict:** what will `ol>li*4` give you? Type it and check.

Quick reference

You want to...	Do this
Start a new page	`!` + Tab, then change the title
Open the page in Chrome	Right-click → Open with Live Server
No Live Server	Double-click the file, then F5 after each save
Fix messy indentation	Shift+Alt+F
See all HTML errors	Ctrl+Shift+M
Rename a tag pair	Edit the opening tag; linked editing changes the closing tag
Copy a line	Alt+Shift+↓

Mistakes everyone makes

You see...	Because...	Fix
The page does not change after you edit	You did not save (autosave is off)	Ctrl+S, every time
Emmet does nothing when you press Tab	The file is not saved as `.html`, or you typed a space inside the abbreviation	Save as `index.html` first; no spaces in `ul>li*3`
The tab says "Document"	You kept Emmet's default title (see D2-03 bug 2 tomorrow)	Write a real title
Live Server opens the wrong page or "Cannot GET"	You opened a parent folder or a single file	File → Open Folder → `W2D1`
No squiggles even on broken HTML	HTMLHint not installed, or you opened a file instead of the folder	Install HTMLHint; open the `W2D1` folder
Closing tag did not rename	Linked editing is off	Open the `W2D1` folder so its settings apply

You see...	Because...	Fix
Two copies of Chrome tabs pile up	Each "Open with Live Server" opens a new tab	Keep one tab; Live Server reloads it for you

Check yourself

1. Why do we open the **folder** `W2D1` and not a single file?
2. What does Live Server do when you press Ctrl+S?
3. Write the Emmet abbreviation for: a `nav` with 4 links inside.
4. What do `>` and `+` mean in Emmet? Give one example of each.
5. What will `.card>h2+p` produce? Write the HTML.
6. Which shortcut fixes indentation? Which one opens the Problems panel?
7. You have no Live Server in class. What do you do?
8. Explain Emmet to a friend using the phone speed-dial idea.

Try it

- **D1-00 Setup check**: open `W2D1`, start Live Server, put your name in the `<h1>`, open F12.
- **D1-02 Emmet speed drill**: make `index.html` match `target.html` using the 6 required abbreviations (`!`, `h1+p`, `ul>li*5`, `nav>a*3`, `table>tr*3>td*2`, `.card>h2+p`) and the 2 bonus ones (`ul>li.item$*3`, `lorem20`).

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. Example prompt:

> "Explain what each symbol means in the Emmet abbreviation `table>tr*3>td*2`. Do not write the HTML for me."

Remember the One Rule: you may only ship code you can explain, line by line. Next: 03 · The HTML skeleton and anatomy.

03 · The HTML Skeleton and Anatomy: Student Notes

File: notes/03-html-skeleton-and-anatomy.md

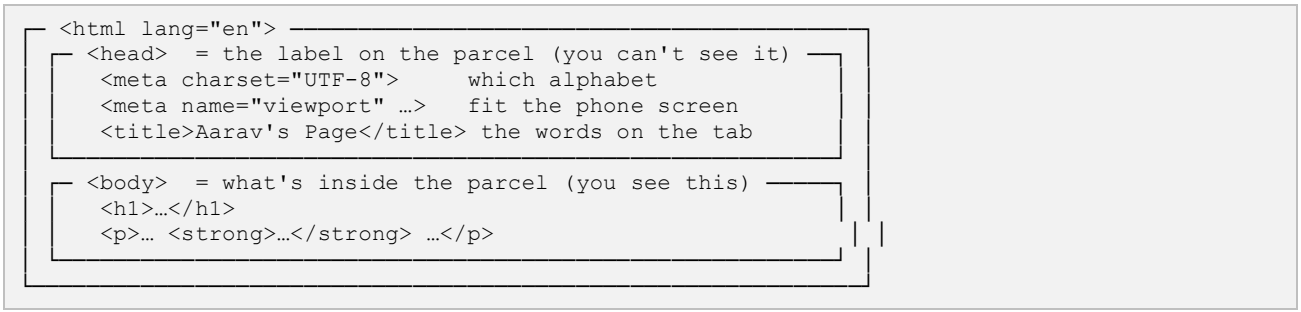
TSI Nepal · AI-Powered Full Stack Developer Program · Week 2, Day 1 · Block B3 · Layer: Frontend

Today in one sentence: Every web page stands on the same skeleton, built from labeled boxes inside boxes.

The picture to keep in your head

Skeleton (nothing stands without it) + **labeled boxes inside boxes**: opening label, contents, closing label. **Last box opened, first box closed.**

And for the two big parts of the page: `<head>` = **the label on the parcel** (information about the page, invisible); `<body>` = **what's inside the parcel**.



Words you now own

Term	What it means (plain English)	Example
HTML	HyperText Markup Language: text with labels that say what each part **is**	`<h1>Momo House</h1>`
Tag	The label itself, in angle brackets	`<p>` (opening), `</p>` (closing)
Element	Opening tag + contents + closing tag, the whole box	`<p>Hello</p>`
Attribute	Extra information inside the opening tag: `name="value"`	`lang="en"`, `src="images/momo.svg"`
Void element	An element with no contents and no closing tag	`<meta>`, `<img>`, ` `, `<hr>`, `<input>`
DOCTYPE	The first line. Tells the browser "use modern HTML rules"	`<!DOCTYPE html>`
`lang`	The language of the page, for screen readers, Google and translators	`<html lang="en">`
`<head>`	Information **about** the page. Not shown inside the page	title, meta, link to CSS
`<body>`	Everything the visitor sees	headings, text, images
`meta charset`	Which set of letters to use. UTF-8 covers English, Nepali (नमस्ते), symbols	`<meta charset="UTF-8">`
`meta viewport`	Makes the page fit a phone screen	`width=device-width, initial-scale=1.0`
`<title>`	The text on the browser tab and in Google results	`<title>Aarav's Page</title>`
Nesting	Putting one element inside another	`<p>I live in <strong>Kathmandu</strong></p>`
Comment	A note for people. The browser ignores it	`<!-- add photo later -->`
Parent / child	The outer box is the parent, the inner box is the child	`<p>` is the parent of `<strong>`

How it works

1. Anatomy of one element

```
   opening tag         contents         closing tag
 ┌──────┴─────────┐ ┌──────┴──────┐ ┌──────┴──────┐
<p class="intro">Namaste, I am Aarav.</p>
   └──┬──┘ └──┬──┘
attribute  attribute
  name       value (always in "double quotes")
```

- The closing tag has a slash: `</p>`.
- Attributes go **only** in the opening tag.
- Tag names are lowercase in this class: `<p>`, not `<P>`.

2. The skeleton, line by line

Line	Why it is there
`<!DOCTYPE html>`	Without it, the browser switches to old "quirks mode" and copies 1990s mistakes. Always the very first line
`<html lang="en">`	The root box. Everything else goes inside it. `lang` tells screen readers to use an English voice
`<head>`	Opens the parcel label
`<meta charset="UTF-8">`	So `Rs.`, `©`, `→` and `नमस्ते` print correctly, not as `Ã©`
`<meta name="viewport" ...>`	So the page is not tiny on a phone
`<title>...</title>`	The tab text. Required. One per page
`</head>`	Closes the label
`<body>` … `</body>`	The contents of the parcel
`</html>`	Closes the root box. **Nothing comes after it**

3. The nesting rule: last opened, first closed

Boxes inside boxes must close in the opposite order they opened, like Russian dolls.

```
RIGHT:  <p> I live in <strong> Kathmandu </strong> </p>
         1            2                 2         1     (open 1, open 2, close 2, close 1)

WRONG:  <p> I live in <strong> Kathmandu </p> </strong>
         1            2                 1     2         (crossed: box 1 closes while box 2 is still open)
```

You can never "cross" boxes. Shift+Alt+F (format) makes crossed boxes easier to spot: the indentation looks wrong.

4. Block and inline (a first look)

- **Block** elements start on a new line and take the full width: `h1`, `p`, `ul`, `table`.
- **Inline** elements sit inside a line of text: `strong`, `em`, `a`, `span`, `img`.
- Rule of thumb: an inline element goes **inside** a block, never the other way round. `<strong><p>…</p></strong>` is wrong.

5. Comments

```
<!-- This is a comment. It opens with <!-- and closes with --> -->
```

A comment must close with `-->` (two dashes). If it does not close, **everything after it becomes part of the comment and disappears**: the footer, the rest of the page, all of it (D1-04 bug 6).

6. Browsers are polite guessers

A browser **never shows an error message** for bad HTML. It guesses what you meant and keeps going. Sometimes the guess is right, so the page looks fine and the bug stays hidden. Sometimes the guess is very wrong: a whole page in bold, a footer that vanished. That is why you need a **spell-checker** (HTMLHint) and the **Elements panel** (F12). Note 10 explains both and lists exactly what the browser repairs.

The 3-step look:

1. **Eyes** (V): does the page look right?
2. **HTMLHint** (D(HH)): any squiggles? Open the Problems panel with Ctrl+Shift+M.
3. **F12 → Elements** (D(F12)): what did the browser actually build? Click the arrows to open each box.

Code you built today

Step 1: DOCTYPE and html lang

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Aarav's Page</title>
</head>
<body>
  <p>The skeleton starts here.</p>
</body>
</html>
```

- `<!DOCTYPE html>` is first. `lang="en"` is on `<html>`.
- **Predict:** what text will the browser tab show?

Step 2: head, meta and title

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aarav's Page</title>
</head>
<body>
  <p>Namaste! नमस्ते! Price: Rs. 150 → good value.</p>
</body>
</html>
```

- `meta charset` makes the Nepali word and the arrow print correctly.
- `meta` is a **void** element: no closing tag.
- **Predict:** delete the `meta charset` line and save. Does the Nepali text still show correctly? (Live Server usually sends UTF-8 anyway; with the double-click fallback you may see strange letters. Put the line back.)

Step 3: body with a heading and a paragraph

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aarav's Page</title>
</head>
<body>
  <h1>Aarav's first page</h1>
  <p>I am a student at Tech Skills Institute in Kathmandu.</p>
  <p>I am learning to build websites.</p>
</body>
</html>
```

- One `<h1>` = the main title of the page.
- Each `<p>` is a paragraph: it starts on a new line with space around it.

Step 4: nesting and comments

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aarav's Page</title>
</head>
<body>
  <h1>Aarav's first page</h1>
  <p>I live in <strong>Kathmandu</strong>, near Dillibazar.</p>
  <p>My favourite food is <strong>momo</strong>, of course.</p>
  <!-- next week: add a photo of Swayambhu -->
  <footer>
    <p>Made by Aarav, Week 2</p>
  </footer>
</body>
</html>
```

- `<strong>` opens and closes **inside** the `<p>`. Last opened, first closed.
- The comment ends with `-->`, so the footer still shows.
- **Predict:** change `-->` to `->` in the comment and save. What disappears? Then put it back.
- Open F12 → Elements. Click the arrow next to `<body>`. You see the boxes inside boxes.

Step 5: the spell-checker demo (squiggles)

This page has three mistakes on purpose. Type it, save it, and press **Ctrl+Shift+M**.

File: `htmlhint-demo.html`

✗ Broken on purpose: find what is wrong

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>HTMLHint demo</title>
</head>
<body>
  <h1>Momo House
  <img src="images/momo.svg">
</body>
</html>
```

- No DOCTYPE → HTMLHint `doctype-first`.
- `<h1>` never closed → `tag-pair`.
- `<img>` with no `alt` → `alt-require`.
- Fix all three until the Problems panel says "No problems".

Quick reference

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>A real title</title>
</head>
<body>
  <h1>One main heading</h1>
  <p>Content.</p>
</body>
</html>
```

Mistakes everyone makes

You see...	Because...	Fix
HTMLHint: "Doctype must be declared first"	No `<!DOCTYPE html>` on line 1 (D1-04 bug 1, D1-HW4 bug 1)	Add it as the first line
HTMLHint: "An lang attribute must be present"	`<html>` has no `lang` (D1-04 bug 2)	`<html lang="en">`
The tab shows the file path, and "Aarav's Page" appears **on the page**	`<titel>` typo: the browser does not know that tag, so it moves it into the body as plain text (D1-04 bug 3)	`<title>…</title>`
Everything under the heading is huge	`<h1>` never closed (D1-04 bug 4)	Add `</h1>`
HTMLHint: "Tag must be paired" on a `</strong>`	Crossed boxes `<p>…<strong>…</p></strong>` (D1-04 bug 5)	Close `</strong>` before `</p>`
The footer and everything after it vanished	A comment ends with `->` instead of `-->` (D1-04 bug 6)	Two dashes: `-->`
A whole page is blank	`<title>` never closed: everything after it became the title (D1-HW1 bug 1)	Close `</title>`
Elements panel shows only one head, your second `<head>` is gone	A second `<head>` instead of `</head>`; the browser ignores it (D1-FF5 bug 1)	`</head>`
Text written after `</html>` still shows	The browser moves it into the body; still wrong (D1-FF5 bug 2)	Put it inside `<body>`
`<title>` inside `<body>`: the tab still shows it	The browser takes the first `<title>` anywhere and hides it; still wrong (D1-HW4 bug 2)	Move it into `<head>`

Check yourself

1. Draw the skeleton from memory: which 10 lines does every page need?
2. What is the difference between a **tag** and an **element**? Give an example.
3. Name four void elements.
4. What goes in `<head>`? What goes in `<body>`? Use the parcel idea.
5. Why does `lang="en"` matter if the page looks the same without it?
6. Is this right or wrong, and why? `<p>Momo is <em>very</p></em> tasty`
7. What happens to the page if a comment is never closed?
8. "Browsers are polite guessers." Explain it to a friend in two sentences.

Try it

- **D1-03 Skeleton from memory**: type the whole skeleton with these notes closed. Wrong is fine. Blank is not.
- **D1-04 Broken Skeleton**: 6 bugs in "Aarav's first page". Use eyes, HTMLHint and F12. Fix bug 4 first if the page looks strange.
- **D1-FF5 Notice Board** (fast finishers): bugs 1 and 2 are skeleton bugs.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. Example prompt:

> "HTMLHint says 'Tag must be paired, missing: [ ]' on line 12 of my page. Explain what this message means. Do not rewrite my code."

Remember the One Rule: you may only ship code you can explain, line by line. Next: 04 · Text tags and entities.

04 · Text Tags and Entities: Student Notes

File: notes/04-text-tags-and-entities.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2, Day 1 · Block B4 · Layer: Frontend

Today in one sentence: Text tags are the tone of your voice: they say what each piece of text **means**, not just how it looks.

The picture to keep in your head

Tone of voice: `strong` = "this matters", `em` = stressed word, `br` = a pause in one thought, `blockquote` = "quoting someone else".

And for entities: **escape codes**. `<` means "print a <, don't start a tag".

When you speak...	In HTML
A title you shout across the room	`<h1>`
A smaller title for one part	`<h2>`, `<h3>` ... `<h6>`
Normal talking, one idea at a time	`<p>`
"Listen, **this matters**"	`<strong>`
"I said *Thursday*, not Friday" (stressed word)	`<em>`
A short breath inside one thought (an address, a poem)	` `
"Now, a new topic" (a change of subject)	`<hr>`
"As Didi always says: ..." (a long quote)	`<blockquote>`
A short quote inside a sentence	`<q>`

Words you now own

Term	What it means (plain English)	Example
Heading	A title for the page or a part of it, level 1 to 6	`<h2>Today's specials</h2>`
Heading hierarchy	Headings go in order like a book: chapter, section, sub-section	h1 → h2 → h3, no jumps
Paragraph	A block of text about one idea	`<p>…</p>`
Semantic	The tag says what the content **means**	`<strong>` = important
Presentational	The tag only says how it **looks**	`<b>` = bold, `<i>` = italic
`<span>`	An inline box with no meaning, used to label a few words (for CSS on Day 2)	`<span class="price">Rs. 150</span>`
Entity	A code that prints a special character	`<` → <
` `	A non-breaking space: the two words never split onto two lines	`Rs. 150`
`©`	The © sign	`© 2026 Momo House`

How it works

1. Headings: one h1, no jumps

Headings build an outline of the page. Screen readers and Google read that outline.

```
h1  TSI Canteen Weekly              ← one per page: the main title
  h2  This week's menu
    h3  Monday
    h3  Tuesday
  h2  News
    h3  New tea machine
```

- **One `<h1>` per page.** It is the title of the whole page (D1-06 bug 1).
- **No jumps.** After an h2 comes h2 or h3, never h5 (D1-FF5 bug 3, D1-15 bug 6).
- **Never pick a heading for its size.** Want bold text? Use `<strong>`. Size comes from CSS on Day 2 (D1-FF5 bug 4).
- These are **R** bugs: no tool in Week 2 flags them. You must read the code against the rule.

2. `strong` vs `b`, `em` vs `i`

Tag	Looks	Means	Use it when
`<strong>`	bold	"this is important"	a warning, a key price, a deadline
`<b>`	bold	nothing, just bold	a product name in a review (rare)
`<em>`	italic	stress on this word	"I said *Thursday*"
`<i>`	italic	a different voice: a foreign word, a ship name	*momo* in an English sentence, *dal bhat*

A screen reader can change its voice for `strong` and `em`. It ignores `b` and `i`. So in this course: **use `strong` and `em` by default.**

3. Line break and divider

- `<br>` = a line break **inside** one thought: an address, a poem. It is a void element (no closing tag).
- Do **not** use `<br><br><br>` to make space. Use two `<p>` elements (D1-FF5 bug 5). Space is CSS's job.
- `<hr>` = a change of topic, drawn as a line. Also void. `</hr>` is not a tag: the browser ignores it, so no line appears (D1-FF5 bug 6).

4. Quotes

- `<blockquote>` for a long quote, on its own. It is indented. Close it with `</blockquote>`, never `</quote>` (D1-06 bug 4).
- `<q>` for a short quote inside a sentence. The browser adds the quote marks for you.

5. Entities: escape codes

Some characters have a job in HTML. `<` starts a tag. `&` starts an entity. To **print** them, use an escape code.

You want to show	Write	Why
<	`<`	"less than": otherwise it starts a tag
>	`>`	"greater than"
&	`&`	otherwise it starts an entity
©	`©`	copyright
a space that never breaks	` `	keeps `Rs. 150` together
→	`→`	"right arrow"
♥	`♥`	heart
"	`"`	a quote mark inside an attribute value

Every entity is `&` + name + `;`. A typo like `&nbps;` is not a known name, so the browser prints it letter by letter (D1-06 bug 3, D1-FF4).

6. HTML ignores extra spaces and Enter

You can type ten spaces or five new lines in your code. The browser shows **one** space. This is called whitespace collapsing. To start a new paragraph you need a new `<p>`, not Enter.

Code you built today

Step 1: headings

File: `headings.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TSI Canteen Weekly</title>
</head>
<body>
  <h1>TSI Canteen Weekly</h1>
  <h2>This week's menu</h2>
  <h3>Sunday</h3>
  <p>Chicken momo and milk tea.</p>
  <h3>Monday</h3>
  <p>Veg chowmein and lemon tea.</p>
  <h2>News</h2>
  <p>Didi has a new tea machine.</p>
</body>
</html>
```

Predict: if you change the second `<h2>` to `<h5>`, will the browser complain? (No. That is why it is an R bug.)

Step 2: strong, em, br, hr

File: `tone.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Tone of voice · TSI Canteen</title>
</head>
<body>
  <h1>Canteen notice</h1>
  <p>The canteen closes at <strong>4 pm sharp</strong> on Thursday.</p>
  <p>I said <em>Thursday</em>, not Friday.</p>
  <hr>
  <h2>Find us</h2>
  <p>TSI Canteen<br>
     Ground floor, Dillibazar<br>
     Kathmandu</p>
</body>
</html>
```

- `<br>` keeps the address as one thought on three lines.
- `<hr>` marks the change of topic.

Step 3: blockquote, q and span

File: `quotes.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Didi says · TSI Canteen</title>
</head>
<body>
  <h1>Didi's words of wisdom</h1>
  <blockquote>
    <p>A good momo needs three things: fresh flour, patience, and a hungry student.</p>
  </blockquote>
  <p>Aarav asked for the recipe. Didi just said <q>come early tomorrow</q>.</p>
  <p>Today's price: <span class="price">Rs. 150</span> a plate.</p>
</body>
</html>
```

- `<span>` changes nothing you can see today. On Day 2, CSS will colour every `.price`.

Step 4: entities

File: `entities.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Entities · TSI Canteen</title>
</head>
<body>
  <h1>HTML lesson in the canteen</h1>
  <p>Wrap text in &amp;lt;p&amp;gt; tags to make a paragraph.</p>
  <p>Momo &amp;amp; tea: Rs.&amp;nbsp;150</p>
  <p>Sunday &amp;rarr; Thursday, we cook with &amp;hearts;</p>
  <p>&amp;copy; 2026 TSI Canteen</p>
</body>
</html>
```

Predict: what will the first paragraph show if you write `<p>` instead of `<p>`? (Try it. The word disappears, because the browser starts a new paragraph instead. That is D1-06 bug 2.)

Step 5: Grandmother's note (D1-05 style)

Plain text in, meaning out:

File: `grandmother.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>A note from Grandmother</title>
</head>
<body>
  <h1>A note from Grandmother</h1>
  <p>Dear Aarav,<br>
     I hope Kathmandu is treating you well.</p>
  <p>Please <strong>eat proper food</strong>, not only momo from the canteen.</p>
  <p>And call your mother on <em>Saturday</em>, not Sunday.</p>
  <blockquote>
    <p>Work hard, sleep early, and the city will be kind to you.</p>
  </blockquote>
</body>
</html>
```

Quick reference

You want...	Write
Main title	`<h1>…</h1>` (once)

You want...	Write
Section title	`<h2>…</h2>`, then `<h3>` inside it
Important	`<strong>…</strong>`
Stressed word	`<em>…</em>`
Line break in one thought	` `
Topic change	`<hr>`
Long quote	`<blockquote><p>…</p></blockquote>`
Short quote	`<q>…</q>`
Label a few words	`<span class="…">…</span>`
< > & © space → ♥	`< > & ©   → ♥`

Mistakes everyone makes

You see...	Because...	Fix
Two main titles on one page	Two `<h1>` (D1-06 bug 1) — R	Keep one h1; the other becomes h2
"Wrap text in tags": the word `<p>` vanished	The browser read `<p>` as a real tag (D1-06 bug 2)	`<p>`
`Rs.&nbps;150` printed literally	Typo in the entity name (D1-06 bug 3, D1-FF4)	` `
The rest of the page is indented	`<blockquote>` closed with `</quote>` (D1-06 bug 4)	`</blockquote>`
Everything after one word is italic	`<em>` never closed (D1-06 bug 5)	Add `</em>`
h2 then h5	Heading jump (D1-FF5 bug 3, D1-15 bug 6) — R	Use the next level only
A sentence in `<h4>` just to make it bold	Heading used for looks (D1-FF5 bug 4) — R	`<p><strong>…</strong></p>`
`   ` for space	br used for layout (D1-FF5 bug 5)	Two `<p>` elements
The divider line is missing	`</hr>` is not a tag (D1-FF5 bug 6)	`<hr>`

Check yourself

1. Why only one `<h1>` per page? Who reads the headings besides you?
2. Is h1 → h2 → h4 correct? What rule does it break? Which tool will tell you? (Careful.)
3. `strong` and `b` both look bold. What is the difference?
4. When should you use `<br>`? When should you **not** use it?
5. Write the entities for `<`, `&`, © and a non-breaking space.
6. **Predict:** what does the browser show for `<p>5 < 10 && 10 > 5</p>`?
7. Why does `Rs. 150` matter on a phone screen?
8. Explain the "tone of voice" idea to a friend, with `strong` and `em` as examples.

Try it

- **D1-05 Grandmother's note**: turn a plain 4-sentence note into HTML with strong, em, br, blockquote.
- **D1-06 Canteen News**: 5 bugs in "TSI Canteen Weekly".
- **D1-FF1 Entity art** (fast finishers): a price board and a paragraph that shows `<p>` literally.
- **D1-FF5 Notice Board**: bugs 3–6 are text bugs.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. Example prompt:

> "What is the difference between `<strong>` and `<b>` for a screen-reader user? Give me a one-paragraph

answer, no code."

Remember the One Rule: you may only ship code you can explain, line by line. Next: 05 · Lists.

05 · Lists: Student Notes

File: notes/05-lists.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2, Day 1 · Block B5 · Layer: Frontend

Today in one sentence: Ask one question before every list: does the order matter?

The picture to keep in your head

Shopping list (order doesn't matter → `ul`) vs **recipe steps** (order matters → `ol`); a nested list = a **sub-list under one item**.

```
SHOPPING LIST (ul)              RECIPE STEPS (ol)
• flour                         1. Make the dough.
• minced chicken                2. Make the filling.
• onion                         3. Fold the momos.
• spices                        4. Steam for 12 minutes.
   ◦ cumin      ← sub-list
   ◦ coriander    under "spices"
```

If you swap two shopping items, nothing changes. If you swap steps 2 and 4, you eat raw dough.

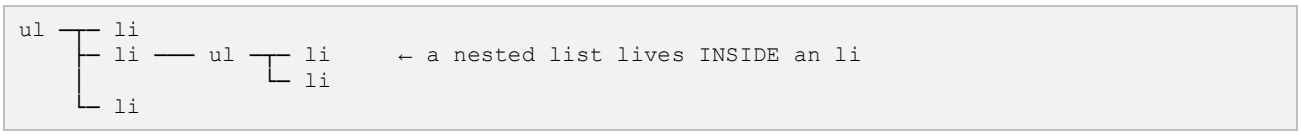
Words you now own

Term	What it means (plain English)	Example
`<ul>`	Unordered list: bullets, order does not matter	a shopping list
`<ol>`	Ordered list: numbers, order matters	recipe steps, a ranking
`<li>`	List item: one entry. Lives **only** inside `ul` or `ol`	`<li>Veg momo</li>`
Nested list	A whole list inside one `<li>`	spices → cumin, coriander
`type` (on `ol`)	Which kind of numbers	`type="A"` → A, B, C · `type="i"` → i, ii, iii
`start` (on `ol`)	The first number	`start="5"` → 5, 6, 7
`<dl>`	Description list: terms and their meanings, like a glossary	momo glossary
`<dt>`	Description term: the word	`<dt>Jhol</dt>`
`<dd>`	Description details: what it means	`<dd>Momo in a soup</dd>`

How it works

1. The family rule

A `<li>` must have a `<ul>` or `<ol>` parent. A `<ul>` or `<ol>` may contain **only** `<li>` elements directly. No bare text, no `<p>`, no other list directly inside it.



Browsers are polite guessers. If you forget the `<ul>`, Chrome does not complain. The items still show, one per line, so the page looks almost right (D1-07 bug 1). The clues: **the list is not indented, and the bullets are gone** (they hang off the left edge of the page, where you can't see them). The Elements panel shows the truth: no `<ul>` box around the items.

2. Nested lists go inside the `<li>`

The sub-list must sit **before** the `</li>` of its parent item:

```
RIGHT:  <li>Spices
          <ul> <li>Cumin</li> <li>Coriander</li> </ul>
        </li>

WRONG:  <li>Spices</li>
        <ul> <li>Cumin</li> <li>Coriander</li> </ul>    ← the sub-list is an orphan between items
```

The wrong version often **looks** right (it is still indented), so this is a **D(F12)** bug (D1-FF5 bug 7, D1-HW1 bug 4).

3. Ordered list options

Attribute	Example	Gives
(none)	`<ol>`	1, 2, 3
`type="A"`	`<ol type="A">`	A, B, C
`type="a"`	`<ol type="a">`	a, b, c
`type="I"`	`<ol type="I">`	I, II, III
`start="4"`	`<ol start="4">`	4, 5, 6
`reversed`	`<ol reversed>`	3, 2, 1 (a countdown)

4. Description lists: a glossary

`<dl>` holds pairs: a `<dt>` (the word) followed by one or more `<dd>` (what it means). Use it for a glossary, an FAQ, or "label: value" details. `<li>` does not belong inside `<dl>` (D1-07 bug 4).

Code you built today

Step 1: ul and ol

File: `lists.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo shopping list and steps</title>
</head>
<body>
  <h1>Didi's momo plan</h1>

  <h2>Shopping list</h2>
  <ul>
    <li>Flour</li>
    <li>Minced chicken</li>
    <li>Onion</li>
    <li>Ginger and garlic</li>
  </ul>

  <h2>Steps</h2>
  <ol>
    <li>Make the dough.</li>
    <li>Make the filling.</li>
    <li>Fold the momos.</li>
    <li>Steam for 12 minutes.</li>
  </ol>
</body>
</html>
```

Predict: change `<ol>` to `<ol type="A" start="3">`. What will the first step be labelled?

Step 2: a nested list

File: `nested.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Nested shopping list</title>
</head>
<body>
  <h1>Shopping list</h1>
  <ul>
    <li>Flour</li>
    <li>Minced chicken</li>
    <li>Spices
      <ul>
        <li>Cumin</li>
        <li>Coriander</li>
        <li>Timur (Sichuan pepper)</li>
      </ul>
    </li>
    <li>Onion</li>
  </ul>
</body>
</html>
```

- The inner `<ul>` opens and closes **between** `Spices` and `</li>`.
- The browser shows the sub-list with a different bullet (◦).
- Open F12 → Elements: the inner `ul` sits inside the `li`. That is what "nested" means.

Step 3: a description list

File: `glossary.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo glossary</title>
</head>
<body>
  <h1>Momo glossary</h1>
  <dl>
    <dt>Steam momo</dt>
    <dd>Momo cooked in steam. Soft and light.</dd>
    <dt>Jhol momo</dt>
    <dd>Momo served in a spicy, sour soup.</dd>
    <dt>Kothey</dt>
    <dd>Half fried, half steamed.</dd>
    <dd>Crispy on one side only.</dd>
  </dl>
</body>
</html>
```

- One `<dt>` can have two `<dd>` (Kothey has two).
- No bullets, no numbers: the details are indented under each term.

Step 4: the full momo recipe (D1-08 style)

Copy `momo-plate.svg` into an `images` folder next to the file.

File: `recipe.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo recipe · Momo House</title>
```

```
</head>
<body>
  <h1>Didi's chicken momo</h1>
  <img src="images/momo-plate.svg" alt="A plate of ten steamed momos with red chutney" width="300">

  <h2>Ingredients</h2>
  <ul>
    <li>2 cups flour</li>
    <li>500 g minced chicken</li>
    <li>Spices
      <ul>
        <li>Cumin</li>
        <li>Coriander</li>
      </ul>
    </li>
    <li>1 onion</li>
  </ul>

  <h2>Steps</h2>
  <ol>
    <li>Mix flour and water into a dough. Rest it for 30 minutes.</li>
    <li>Mix the chicken, onion and spices.</li>
    <li>Roll small circles of dough and fold in the filling.</li>
    <li>Steam for 12 minutes.</li>
  </ol>

  <h2>Words</h2>
  <dl>
    <dt>Achar</dt>
    <dd>The tomato chutney served with momo.</dd>
  </dl>
</body>
</html>
```

Predict: if you move `</li>` from after the inner `</ul>` to right after the word `Spices`, does the page look different? Check in the browser, then in the Elements panel.

Quick reference

You want…	Write
Bullets	`<ul><li>…</li></ul>`
Numbers	`<ol><li>…</li></ol>`
Letters / Roman numbers	`<ol type="A">`, `<ol type="i">`
Start at 5	`<ol start="5">`
Sub-list	`<li>Item <ul><li>…</li></ul></li>`
Glossary	`<dl><dt>Word</dt><dd>Meaning</dd></dl>`
Emmet	`ul>li*4`, `ol>li*3`, `dl>(dt+dd)*3`

Mistakes everyone makes

You see…	Because…	Fix
Items with **no indent and no bullets**	`<li>`s with no `<ul>` around them (D1-07 bug 1, D1-15 bug 5) — V + D(F12)	Wrap them in `<ul>`
Steps shown with bullets	Order matters but you used `ul` (D1-07 bug 2)	`<ol>`
One item has no bullet; HTMLHint complains	`<il>` typo (D1-07 bug 3)	`<li>`
Glossary meanings show bullets	`<li>` used inside `<dl>` (D1-07 bug 4)	`<dd>`
Sub-list looks fine but F12 shows it between items	Nested list placed after `</li>` (D1-FF5 bug 7, D1-HW1 bug 4) — D(F12)	Move it inside the `<li>`
HTMLHint: tag must be paired	`<ol>` closed with `</ul>` (D1-FF5 bug 8)	`</ol>`
A heading word appears inside the list box	Bare text "Notices:" directly inside `<ul>` (D1-FF5 bug 9)	Put it in an `<h2>` or `<p>` before the list

Check yourself

1. What single question decides `ul` or `ol`?
2. Give two real examples of each, from your own day.
3. Where exactly does a nested list go? Write the lines.
4. Why can a missing `<ul>` hide from your eyes? What clue gives it away?
5. What do `type="I"` and `start="10"` do on an `ol`?
6. What is a `dl` for? Which two tags live inside it?
7. **Predict:** what does `<ol reversed><li>a</li><li>b</li><li>c</li></ol>` show?
8. Explain to a friend why a recipe is an `ol` but a shopping list is a `ul`.

Try it

- **D1-07 Broken Lists**: 4 bugs. Hint for bug 1: look for the missing indent (and the missing bullets).
- **D1-08 Momo recipe**: ingredients `ul` with one nested sub-list inside an `li`, steps `ol`, one image with alt. Bonus: a `dl` glossary.
- **D1-FF5 Notice Board**: bugs 7–9 are list bugs.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. Example prompt:

> "Why does my nested `<ul>` have to be inside the `<li>` and not after it, if both look the same in Chrome? Explain simply, no code."

Remember the One Rule: you may only ship code you can explain, line by line. Next: 06 · Links and paths.

06 · Links and Paths: Student Notes

File: notes/06-links-and-paths.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2, Day 1 · Block B6 (part 1) · Layer: Frontend

Today in one sentence: A link is an address. Write the address from where **you** are standing.

The picture to keep in your head

Same house vs postal address: relative = "the kitchen", absolute = full postal address. `../` = **"go downstairs one floor"**.

- Inside your own house you say "get it from the kitchen". You don't say "House 14, Dillibazar, Kathmandu, Nepal, the kitchen". That is a **relative** path: it starts from where you are.
- To send a letter to another city you need the **full postal address**. That is an **absolute** URL: `https://www.wikipedia.org`.
- If you are upstairs in `pages/` and want something on the ground floor, first **go downstairs one floor**: `../`.

```
momo-house/                ← ground floor (the site root)
├── index.html
├── images/
│   ├── logo-momo-house.svg
│   └── thamel.svg
└── pages/                 ← first floor
    ├── about.html
    └── menu.html


From index.html          to pages/about.html   →  href="pages/about.html"     (go up the stairs into pages)
From pages/about.html    to index.html          →  href="../index.html"        (go downstairs one floor)
From pages/about.html    to images/thamel.svg   →  src="../images/thamel.svg"  (downstairs, then into images)
From pages/about.html    to pages/menu.html     →  href="menu.html"            (same floor, next room)
```

Words you now own

Term	What it means (plain English)	Example
`<a>`	Anchor: a link	`<a href="menu.html">Menu</a>`
`href`	Hypertext reference: where the link goes	`href="pages/about.html"`
Link text	The words the visitor clicks. Must say where it goes	"See the menu", never "click here"
Absolute URL	A full address with `https://`	`https://www.wikipedia.org`
Relative path	An address that starts from the current file's folder	`pages/menu.html`, `../index.html`
`../`	Go downstairs one folder. Other people call this "go up one folder"; it is the same thing.	`../images/momo.svg`
Site root	The top folder of the site	`W2D1/classwork/D1-09-broken-paths/`
`target="_blank"`	Open the link in a new tab	for links to other websites
`mailto:`	A link that opens an email	`href="mailto:tour@tsi.edu.np"`
`tel:`	A link that phones a number (on a phone)	`href="tel:+9779800000000"`
Fragment / anchor	`#` + an id: jump to a spot on a page	`href="#menu"` → `id="menu"`
404	"Not Found": nothing at that address	a typo in `href`

How it works

1. Anatomy of a link

```
<a href="pages/menu.html" target="_blank">See today's menu</a>
   └──────────┬──────────┘ └──────┬──────┘ └──────┬──────┘
      where it goes          new tab (optional)   what the visitor clicks
```

2. Absolute or relative?

Going to...	Use	Example
Another website	absolute, with `https://`	`https://www.wikipedia.org`
A page in your own site	relative	`menu.html`, `pages/about.html`
An image in your own site	relative	`images/momo.svg`

Never use a path from your own computer like `C:\Users\Aarav\Desktop\site\index.html`. It works only on Aarav's laptop. On anyone else's computer, and on a real server, that path does not exist (D1-09 bug 6).

Never forget `https://` on an external link. `href="www.wikipedia.org"` has no scheme, so the browser thinks it is a **file in your folder** named `www.wikipedia.org` → 404 (D1-09 bug 3, D1-15 bug 10).

3. How the browser reads a relative path

The browser starts **in the folder of the current page** and follows the path one step at a time.

```
Page:  pages/about.html      Path: ../images/thamel.svg
Start in  pages/
../       → go down to the root
images/   → into the images folder
thamel.svg → the file.  ✔
```

If the page is `index.html` (already on the ground floor) and you write `../images/…`, you walk out of the site (D1-15 bug 9). If the page is `pages/about.html` and you write `images/thamel.svg`, the browser looks for `pages/images/thamel.svg`, which does not exist (D1-09 bug 5).

4. File names: exact, lowercase, no spaces

- `menu.html` is not `menu.htm` (D1-15 bug 1) and not `menu` (D1-HW1 bug 3).
- `images/` is not `image/` (D1-09 bug 1).
- **Case matters on real servers.** Windows treats `Everest.svg` and `everest.svg` as the same file, so the picture shows on your laptop. Most web servers run Linux, where they are **different files**, so the picture breaks once you go live (D1-HW1 bug 2, tag **H**). Rule: always lowercase, words joined with `-`.
- **No spaces.** `notice-board .svg` (with a space) is a different name (D1-FF5 bug 10).

5. Jump links: `#id`

```
<a href="#specials">Jump to specials</a>     ...     <h2 id="specials">Today's specials</h2>
                └──────── must match exactly, without the # ────────┘
```

- `#top` on its own jumps to the top of the page (browsers support it even with no `id="top"`, but in this class we add `id="top"` to the `h1` so the link has a clear target).
- You can jump to a spot on **another** page: `href="treks.html#prices"`. The id must exist on that page (D1-HW1 bug 7, D1-FF5 bug 11).
- An `id` must be unique on a page. Two elements with the same id is an error (D1-FF4).

6. Links must close

`<a>` must end with `</a>`. If you forget it in a nav, the next `<a>` closes it for you, so it looks fine. But if the **last** link is not closed, everything after it (even the `<h1>`) becomes part of that link (D1-15 bug 3). Note 10 shows this in Chrome.

7. New tab, email, phone

- `target="_blank"`: use it for links that leave your site, so your page stays open.
- `mailto:` opens the visitor's email app. `tel:` lets a phone call the number. Write the number with the country code: `+977`.

Code you built today

Step 1: an external link

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House · Home</title>
</head>
<body>
  <h1>Momo House</h1>
  <p>Read about momo on <a href="https://en.wikipedia.org/wiki/Momo_(food)" target="_blank">Wikipedia</a>.</p>
</body>
</html>
```

Predict: delete `https://` and click the link. What does the URL bar show? What status?

Step 2: a small site with folders and `../`

Build this whole folder. All three pages work together.

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House · Home</title>
</head>
<body>
  <img src="images/logo-momo-house.svg" alt="Momo House logo" width="160">
  <h1>Momo House</h1>
  <nav>
    <a href="index.html">Home</a>
    <a href="pages/about.html">About</a>
    <a href="pages/menu.html">Menu</a>
  </nav>
  <p>The best momo in Kathmandu.</p>
</body>
</html>
```

File: `pages/about.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House · About</title>
</head>
<body>
  <h1>About Momo House</h1>
  <nav>
    <a href="../index.html">Home</a>
    <a href="menu.html">Menu</a>
  </nav>
  <p>Didi opened Momo House in Dillibazar in 2015.</p>
  <img src="../images/thamel.svg" alt="A busy street in Thamel with shop signs" width="300">
</body>
</html>
```

File: `pages/menu.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House · Menu</title>
</head>
<body>
  <h1>Menu</h1>
  <nav>
    <a href="../index.html">Home</a>
    <a href="about.html">About</a>
  </nav>
  <p>Steam, fried and jhol momo.</p>
</body>
</html>
```

- `index.html` is on the ground floor: `pages/about.html` goes up into `pages`.
- `about.html` is upstairs: `../index.html` goes downstairs one floor.
- `about.html` → `menu.html`: same floor, no folder needed.
- **Predict:** in `about.html`, change `../images/thamel.svg` to `images/thamel.svg`. Where does the browser look? Open F12 → Network to see the 404.

Step 3: anchors, mailto and tel

File: `contact.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House · Contact</title>
</head>
<body>
  <h1 id="top">Contact Momo House</h1>
  <nav>
    <a href="#hours">Opening hours</a>
    <a href="#find-us">Find us</a>
  </nav>

  <h2 id="hours">Opening hours</h2>
  <p>Sunday to Friday, 10 am to 9 pm. Closed on Saturday.</p>
  <p>Lots of text here so the page can scroll.</p>
  <p>More text.</p>
  <p>Even more text.</p>

  <h2 id="find-us">Find us</h2>
  <p>Dillibazar, Kathmandu.</p>
  <p>Email: <a href="mailto:didi@momohouse.com.np">didi@momohouse.com.np</a></p>
  <p>Phone: <a href="tel:+9779800000000">+977 980 000 0000</a></p>
```

```
  <p><a href="#top">Back to top</a></p>
</body>
</html>
```

- `href="#hours"` jumps to `id="hours"`. No space, same spelling, no `#` in the `id`.
- Make the browser window short so you can see the jump.

Quick reference

You want…	Write
Another website, new tab	`<a href="https://…" target="_blank">Name of site</a>`
Page in the same folder	`<a href="menu.html">Menu</a>`
Page in a sub-folder	`<a href="pages/about.html">About</a>`
Page one floor down	`<a href="../index.html">Home</a>`
Jump on the same page	`<a href="#menu">Menu</a>` + `id="menu"`
Jump on another page	`<a href="treks.html#prices">Prices</a>`
Email	`<a href="mailto:name@site.com">…</a>`
Phone	`<a href="tel:+977…">…</a>`

Mistakes everyone makes

You see…	Because…	Fix
Broken image icon	Folder is `images`, path says `image/` (D1-09 bug 1)	`images/…`
404 when clicking About	`about.html` is inside `pages/` (D1-09 bug 2)	`pages/about.html`
404 with `…/www.wikipedia.org` in the URL bar	No `https://` (D1-09 bug 3, D1-15 bug 10)	`https://www.wikipedia.org`
"Home" link from a sub-page breaks	It needs to go down a floor (D1-09 bug 4)	`../index.html`
Image breaks only on a sub-page	Path starts from the wrong folder (D1-09 bug 5)	`../images/…`
A link works only on one laptop	A `C:\…` path (D1-09 bug 6)	A relative path
404 on `menu.htm` / `treks`	Wrong or missing extension (D1-15 bug 1, D1-HW1 bug 3)	`menu.html`, `treks.html`
Image breaks when the next page is in the same folder	`../images/` used from the root (D1-15 bug 9)	`images/…`
The jump link does nothing	`href="#events"` but `id="event"` (D1-FF5 bug 11, D1-HW1 bug 7)	Make them match
The `<h1>` became a link	The last nav `<a>` was never closed (D1-15 bug 3)	Add `</a>`
Works on Windows, breaks online	`Everest.svg` vs `everest.svg` (D1-HW1 bug 2) — H	Match the case exactly; use lowercase
Image breaks, name looks right	A space in the name (D1-FF5 bug 10)	No spaces in file names

Check yourself

1. Absolute vs relative: which one do you use for Wikipedia? For your own `menu.html`? Why?
2. You are in `pages/menu.html`. Write the path to `index.html` and to `images/momo.svg`.
3. What does the browser do with `href="www.google.com"`? Why?
4. Why is `C:\Users\Aarav\Desktop\site\index.html` a bad link even if it works for Aarav?
5. Why can `Momo.svg` vs `momo.svg` work on your laptop and fail on a real server?
6. Write a link that jumps to `<h2 id="prices">` on the same page.
7. When should you use `target="_blank"`?
8. Explain `../` to a friend using the house idea.

Try it

- **D1-09 Broken Paths**: 6 path bugs across 3 pages. Keep F12 → Network open: every red 404 row is a clue.
- **D1-FF2 Table of contents** (fast finishers): a long page with jump links and "back to top".
- **D1-FF6 Gallery detail page**: a `places/` sub-page with `../index.html`, `../images/` and `mailto:`.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. Example prompt:

> "My page is `pages/about.html` and my image is `images/thamel.svg` in the root folder. Explain step by step how the browser follows a relative path. Do not write the path for me."

Remember the One Rule: you may only ship code you can explain, line by line. Next: 07 · Images.

07 · Images: Student Notes

File: notes/07-images.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2, Day 1 · Block B6 (part 2) · Layer: Frontend

Today in one sentence: An image is a sticker you put on the page, with a description for anyone who cannot see it.

The picture to keep in your head

A sticker, not an envelope (void tag). alt = **the description you read over the phone**.

- An **envelope** (like `<p>…</p>`) holds things inside it, so it needs an opening and a closing.
- A **sticker** is stuck on in one go. Nothing goes inside it. So `<img>` has **no closing tag**.
- Imagine you call your friend and describe the Momo House page over the phone. For the logo you'd say "the Momo House logo". For a photo you'd say "a plate of ten steamed momos with red chutney". **That sentence is the `alt` text.**

Words you now own

Term	What it means (plain English)	Example
`<img>`	Puts an image on the page. A void element	`<img src="images/momo.svg" alt="A momo">`
`src`	Source: the path to the image file (same path rules as links, note 06)	`src="images/momo.svg"`
`alt`	Alternative text: what the image shows, in words	`alt="A plate of momos"`
`alt=""`	Empty alt: "this image is decoration, skip it"	a dhaka border pattern (Day 2)
`width` / `height`	The size in pixels, numbers only	`width="300"`
`<figure>`	A box for an image (or table, or code) plus its caption	a gallery item
`<figcaption>`	The caption under the figure	"Swayambhu at sunrise"
SVG	An image format made of shapes, sharp at any size. Our class images are SVG	`momo.svg`
PNG / JPG	Photo formats made of pixels	`photo.jpg`
Screen reader	Software that reads the page aloud for blind visitors	Windows Narrator, NVDA

How it works

1. Who reads `alt`?

- **Screen readers** read it aloud to visitors who cannot see the image.
- **The browser** shows it when the image fails to load (a wrong path, slow internet).
- **Google** uses it to understand what the picture is.
- HTMLHint flags a missing `alt` (`alt-require`), so it is a **D(HH)** bug (D1-15 bug 2, D1-FF4, D1-HW4 bug 4).

2. Writing good alt text

Image	Bad alt	Good alt
Momo House logo	`logo.svg`, `image`	`Momo House logo`
A photo of a momo plate	`momo`	`A plate of ten steamed momos with red chutney`
Swayambhu stupa	`picture of temple`	`Swayambhu stupa on its hill above Kathmandu`
Chef Didi	`didi.svg`	`Didi, the owner, smiling in her kitchen`
A decorative line	(long description)	`alt=""` (empty: skip it)

Rules:

- Say what is **in** the picture and why it matters. Don't start with "image of".
- Short: one sentence.
- Pure decoration gets `alt=""`, not a missing alt.

3. Size

- `width="300"` = 300 pixels wide. Write only the number (no `px` in HTML attributes).
- If you set only `width`, the browser keeps the shape and works out the height.
- If you set both and they don't match the shape, the image **stretches**. Day 2 shows this with a round avatar.
- On Day 2 you will set sizes with CSS instead.

4. `figure` and `figcaption`

A `<figure>` groups an image and its caption, so a screen reader knows they belong together. The caption can be **shown** to everyone; `alt` is for people who can't see the image. You often need both.

5. Where image paths go wrong

Images use the same path rules as links (note 06). Check, in this order:

1. Is the **folder** right? `images/` not `image/`.
2. Is the **file name** exact, with the right case and no spaces?
3. Are you on the right **floor**? From `pages/` you need `../images/`.
4. Open F12 → **Network**: a red **404** row shows the exact path the browser tried.

Code you built today

Step 1: one image with alt and width

Copy `momo.svg` and `thamel.svg` into an `images` folder next to the page.

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House · Images</title>
</head>
<body>
  <h1>Momo House</h1>
  <img src="images/momo.svg" alt="A single steamed momo" width="200">
  <p>Our momos are folded by hand every morning.</p>
  <img src="images/thamel.svg" alt="A busy street in Thamel with shop signs" width="400">
</body>
</html>
```

- `<img>` has no `</img>`. It is a sticker.
- **Predict:** change `momo.svg` to `momos.svg`. What does the page show in its place? (Your alt text, next to a broken-image icon.)

Step 2: the Kathmandu gallery (D1-10 style)

Copy `swayambhu.svg` and `patan-durbar.svg` into `images`.

File: `gallery.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Kathmandu Gallery</title>
</head>
<body>
  <h1 id="top">Kathmandu Gallery</h1>
  <p>Three places every TSI student should visit.</p>


  <figure>
    <img src="images/swayambhu.svg" alt="Swayambhu stupa on its hill above Kathmandu" width="400">
    <figcaption>Swayambhu, also called the Monkey Temple.</figcaption>
  </figure>


  <figure>
    <img src="images/patan-durbar.svg" alt="The old temples of Patan Durbar Square" width="400">
    <figcaption>Patan Durbar Square, the city of fine arts.</figcaption>
  </figure>


  <p>Learn more about <a href="https://en.wikipedia.org/wiki/Kathmandu" target="_blank">Kathmandu on Wikipedia</a>.</p>
  <p><a href="#top">Back to top</a></p>
</body>
</html>
```

- Each `<figure>` holds one image and one caption.
- **Predict:** what will a screen reader say for the first figure? (The alt, then the caption.)

Step 3: images that are links

An image can sit inside a link. Then the alt must say where the link goes.

File: `logo-link.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House · Logo link</title>
</head>
<body>
  <a href="logo-link.html">
    <img src="images/logo-momo-house.svg" alt="Momo House home page" width="160">
  </a>
  <h1>Logo as a link</h1>
  <p>Click the logo to come back to this page.</p>
</body>
</html>
```

Quick reference

You want...	Write
An image	`<img src="images/momo.svg" alt="…" width="300">`
A decoration	`<img src="images/line.svg" alt="">`
Image + caption	`<figure><img …><figcaption>…</figcaption></figure>`
Image from a sub-page	`<img src="../images/momo.svg" alt="…">`
A clickable logo	`<a href="index.html"><img src="…" alt="Momo House home page"></a>`

Mistakes everyone makes

You see...	Because...	Fix
HTMLHint: "An alt attribute must be present"	`<img>` has no `alt` (D1-15 bug 2, D1-FF4, D1-HW4 bug 4) — D(HH)	Add a short, real description
A broken-image icon	Wrong folder, name or floor (D1-09 bugs 1 and 5, D1-15 bug 9)	Check F12 → Network for the 404 path
Shows on your laptop, broken online	`Room-Deluxe.svg` vs `room-deluxe.svg` (D1-HW4 bug 3, D1-HW1 bug 2) — H	Match the case; use lowercase names

You see...	Because...	Fix
Broken, name looks right	Space in the name (D1-FF5 bug 10)	Rename without spaces
`</img>` in your code	An image is a sticker: no closing tag	Delete `</img>`
`width="300px"`	HTML size attributes take numbers only	`width="300"`
`alt="image"` or `alt="momo.svg"`	That tells the listener nothing	Describe what is in the picture

Check yourself

1. Why does `<img>` have no closing tag? Use the sticker idea.
2. Who uses the `alt` text? Name three.
3. Write good alt text for: the Momo House logo; a photo of Boudha stupa; a decorative border.
4. What is the difference between `alt` and `<figcaption>`?
5. You are in `places/swayambhu.html`. Write the `src` for `images/swayambhu.svg` in the root.
6. What happens if you set `width` and `height` to a shape that doesn't match the image?
7. **Predict:** what shows on the page when `src` is wrong but `alt` is good?
8. Explain alt text to a friend using the "read it over the phone" idea.

Try it

- **D1-10 Kathmandu gallery**: 2 `figure`s with figcaption + alt + width, 1 external link with `target="_blank"`, 1 "back to top" `#top` anchor. Then show & tell.
- **D1-FF3 Mystery page** (fast finishers): rebuild a page from a picture (`target.svg`).
- **D1-FF6 Gallery detail page**: `../images/` from a sub-page.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. Example prompt:

> "Here is my alt text for a photo of Patan Durbar Square: 'image'. Tell me why it is weak and what a good alt text should include. Don't write it for me."

Remember the One Rule: you may only ship code you can explain, line by line. Next: 08 · Tables.

08 · Tables: Student Notes

File: notes/08-tables.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2, Day 1 · Block B7 · Layer: Frontend

Today in one sentence: A table is for data in rows and columns, like the canteen price board, never for page layout.

The picture to keep in your head

Four tags, one team / the **canteen price board**: data, not layout.

```
          TSI CANTEEN PRICE BOARD            ← <caption>
┌──────────────┬──────────┬──────────┐
│ Item         │ Half     │ Full     │     ← a row <tr> of header cells <th>
├──────────────┼──────────┼──────────┤
│ Steam momo   │ Rs. 90   │ Rs. 150  │     ← a row <tr> of data cells <td>
│ Fried momo   │ Rs. 110  │ Rs. 180  │
│ Milk tea     │ Rs. 30 (one size)   │     ← one cell spread over 2 columns: colspan="2"
└──────────────┴──────────┴──────────┘
```

The team	Its job
`<table>`	The whole board
`<tr>`	**T**able **r**ow: one line across the board
`<th>`	**T**able **h**eader cell: the title of a column (or row), bold and centred
`<td>`	**T**able **d**ata cell: one value

Words you now own

Term	What it means (plain English)	Example
Row	A line of cells across	`<tr>…</tr>`
Column	A line of cells down. HTML has no column tag: columns come from cell position	the "Price" column
Cell	One box: `th` or `td`	`<td>Rs. 150</td>`
`<caption>`	The title of the table, right after `<table>`	`<caption>TSI Canteen prices</caption>`
`<thead>`	The group of header rows	the row with "Item, Price"
`<tbody>`	The group of data rows	all the food rows
`<tfoot>`	The group of summary rows	a "Total" row
`colspan`	One cell stretches across several columns	`<td colspan="5">Lunch</td>`
`rowspan`	One cell stretches down several rows	`<td rowspan="2">Project lab</td>`
`scope`	Says what a `th` is the header for: `col` or `row`	`<th scope="col">Price</th>`

How it works

1. The counting rule

Every row must add up to the same number of columns. A normal cell counts 1. A cell with `colspan="3"` counts 3.

```
6 columns (Time + Sun, Mon, Tue, Wed, Thu):
<tr> th + td + td + td + td + td </tr>     = 1+5 = 6      ✔
<tr> th + td + td + td + td </tr>          = 1+4 = 5      ✗ the row is short; a gap on the right
<tr> th + td colspan="5" </tr>             = 1+5 = 6      ✔ (the Lunch row)
<tr> th + td colspan="2" </tr>             = 1+2 = 3      ✗ colspan too small: a gap at the end
```

2. The rowspan rule

A cell with `rowspan="2"` **also takes a seat in the row below**. So the row below must have **one cell less**.

```
            Sun          Mon
10:00   Project lab    HTML        ← "Project lab" has rowspan="2"
11:00   (taken ↑)      CSS         ← this row writes only 2 cells: time + CSS
```

If the row below still writes its own cell, there is one cell too many and it sticks out on the right (D1-11 bug 4, D1-HW1 bug 6).

3. `th` vs `td`

- `th` = a title cell. Bold, centred, and screen readers announce it with each value ("Price: Rs. 150").
- `td` = a value.
- A header row made of `td` looks plain and loses that meaning (D1-11 bug 1, D1-HW1 bug 5).
- Add `scope="col"` to column headers and `scope="row"` to row headers. It tells screen readers which way the header points.

4. Everything must be inside a cell

Text, links, images: all go **inside** a `th` or `td`. Text written between `</tr>` and `<tr>` is outside every cell. The browser cannot leave it there, so it **moves it above the table** (D1-11 bug 5). Cells written directly in `<table>` with no `<tr>`: the browser invents a `<tr>` for them (D1-FF5 bug 12, D1-HW4 bug 5). Note 10 shows both in Chrome.

5. Caption first

`<caption>` must be the first thing inside `<table>`. If it comes after a row, Chrome still shows it, but it is invalid HTML (D1-HW4 bug 6, D(W3C)).

6. Data, not layout

Old websites used tables to place things side by side. Don't. Tables are for **data**. Layout is CSS's job (flexbox and grid, session 3.2).

Code you built today

Step 1: the basic price board

File: `prices.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TSI Canteen · Price board</title>
</head>
<body>
  <h1>TSI Canteen</h1>
  <table>
    <tr>
      <th scope="col">Item</th>
      <th scope="col">Price</th>
    </tr>
    <tr>
      <td>Steam momo</td>
      <td>Rs. 150</td>
    </tr>
    <tr>
      <td>Fried momo</td>
      <td>Rs. 180</td>
    </tr>
    <tr>
```

```
      <td>Jhol momo</td>
      <td>Rs. 200</td>
    </tr>
  </table>
</body>
</html>
```

- Emmet: `table>tr*4>td*2`, then change the first row's `td` to `th`.
- There are no borders yet. Borders are CSS (Day 2). F12 → hover each `<td>` to see the cells.

Step 2: caption, thead, tbody, tfoot

File: `price-board.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TSI Canteen · Full price board</title>
</head>
<body>
  <h1>TSI Canteen</h1>
  <table>
    <caption>Price board, Week 2</caption>
    <thead>
      <tr>
        <th scope="col">Item</th>
        <th scope="col">Half plate</th>
        <th scope="col">Full plate</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <th scope="row">Steam momo</th>
        <td>Rs. 90</td>
        <td>Rs. 150</td>
      </tr>
      <tr>
        <th scope="row">Fried momo</th>
        <td>Rs. 110</td>
        <td>Rs. 180</td>
      </tr>
    </tbody>
    <tfoot>
      <tr>
        <td colspan="3">Prices include tax.</td>
      </tr>
    </tfoot>
  </table>
</body>
</html>
```

- Here each food name is a **row header** (`th scope="row"`).
- The footer row has one cell with `colspan="3"`: 3 = 3 columns. ✔
- **Predict:** change `colspan="3"` to `colspan="4"`. What happens on the right side?

Step 3: colspan and rowspan (the timetable)

File: `timetable.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TSI Week 2 timetable</title>
</head>
<body>
  <h1>TSI Week 2 timetable</h1>
  <table>
    <caption>Morning batch</caption>
```

```
    <thead>
      <tr>
        <th scope="col">Time</th>
        <th scope="col">Sun</th>
        <th scope="col">Mon</th>
        <th scope="col">Tue</th>
        <th scope="col">Wed</th>
        <th scope="col">Thu</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <th scope="row">7:00</th>
        <td>HTML</td>
        <td>HTML</td>
        <td rowspan="2">Project lab</td>
        <td>CSS</td>
        <td>CSS</td>
      </tr>
      <tr>
        <th scope="row">8:00</th>
        <td>Lists</td>
        <td>Tables</td>
        <td>Box model</td>
        <td>Display</td>
      </tr>
      <tr>
        <th scope="row">9:00</th>
        <td colspan="5">Lunch</td>
      </tr>
    </tbody>
  </table>
</body>
</html>
```

Count every row: it must be 6 (Time + 5 days).

- Row 7:00: 1 + 5 = 6 ✔ ("Project lab" counts 1 here)
- Row 8:00: 1 + 4 = 5 cells written + 1 seat taken by "Project lab" = 6 ✔
- Row 9:00: 1 + 5 (colspan) = 6 ✔

Predict: add a 5th `<td>` to the 8:00 row. Where does it appear?

Step 4: the exam results table (D1-12 style)

File: `results.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TSI Week 2 quiz results</title>
</head>
<body>
  <h1>Week 2 quiz results</h1>
  <table>
    <caption>HTML quiz, out of 20</caption>
    <thead>
      <tr>
        <th scope="col">Student</th>
        <th scope="col">HTML</th>
        <th scope="col">Forms</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>Aarav</td>
        <td>14</td>
        <td>12</td>
      </tr>
      <tr>
        <td>Sunita Gurung</td>
        <td>18</td>
```

```
        <td>19</td>
      </tr>
    </tbody>
    <tfoot>
      <tr>
        <td colspan="2">Total</td>
        <td>63</td>
      </tr>
    </tfoot>
  </table>
</body>
</html>
```

- Total row: `colspan="2"` + 1 cell = 3 columns ✔. 14 + 12 + 18 + 19 = 63.

Quick reference

```
<table>
  <caption>Title</caption>
  <thead> <tr> <th scope="col">A</th> <th scope="col">B</th> </tr> </thead>
  <tbody> <tr> <td>1</td> <td>2</td> </tr> </tbody>
  <tfoot> <tr> <td colspan="2">Note</td> </tr> </tfoot>
</table>
```

You want…	Write
A cell over 3 columns	`<td colspan="3">`
A cell down 2 rows	`<td rowspan="2">` (and one cell less in the next row)
A column header	`<th scope="col">`
A row header	`<th scope="row">`
A table quickly	Emmet `table>tr*3>td*4`

Mistakes everyone makes

You see…	Because…	Fix
Header row is not bold	`td` used for headers (D1-11 bug 1, D1-HW1 bug 5)	`th scope="col"`
Cells shifted left; a gap on the right	A row has 4 day cells, not 5 (D1-11 bug 2)	Add the missing cell
"Lunch" fills only one column	`colspam` typo: unknown attribute, ignored (D1-11 bug 3)	`colspan`
An extra cell sticks out on the right	A rowspan cell already takes that seat (D1-11 bug 4, D1-HW1 bug 6)	Delete one cell in the next row
"Tea break 3:00" appears **above** the table	Text outside any cell (D1-11 bug 5)	Put it in a `<td colspan="…">` in its own row
A gap at the end of one row	`colspan="2"` where the row needs 3 columns (D1-15 bug 4). A colspan that is too *big* is almost invisible in Chrome (the extra column is 0 px wide)	`colspan="3"`
F12 shows a `<tr>` you never wrote	Cells directly in `<table>` (D1-FF5 bug 12, D1-HW4 bug 5) — D(F12)	Wrap them in `<tr>`
W3C: caption in the wrong place	`<caption>` after the first row (D1-HW4 bug 6) — D(W3C)	Move it right after `<table>`

Check yourself

1. Name the four team tags and the job of each.
2. Why does HTML have no "column" tag? Where do columns come from?
3. A table has 4 columns. A row has `<th>` + `<td colspan="2">`. Is the row complete? What is missing?
4. A cell has `rowspan="3"`. What must the next two rows do?
5. Why use `th` and not a bold `td`?

6. What does `scope="col"` tell a screen reader?
7. **Predict:** where does the browser put text that you write between two `<tr>`s?
8. Explain to a friend why a table is a price board and not a way to arrange a page.

Try it

- **D1-11 Broken Timetable**: 5 bugs. Count the cells in every row.
- **D1-12 Exam results table**: add `caption`, `thead`/`tbody`, `th scope="col"` and a Total row with `colspan`. Bonus: `tfoot`.
- **D1-FF5 Notice Board**: bug 12 is a table bug.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. Example prompt:

> "My table has 6 columns and one cell has rowspan='2'. Explain how many cells the next row should have and why. No code."

Remember the One Rule: you may only ship code you can explain, line by line. Next: 09 · Forms.

09 · Forms: Student Notes

File: notes/09-forms.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2, Day 1 · Block B8 · Layer: Frontend (the door to the Backend)

Today in one sentence: A form is how the visitor talks back to you.

The picture to keep in your head

The order slip: boxes to fill, ticks, "choose one" circles; label = the printed word next to each box; name = **the column the kitchen writes your answer in**.

```
┌──────────── MOMO HOUSE ORDER SLIP ────────────┐        <form>
│ Name:  [ Aarav_________ ]                     │          <label> + <input type="text" name="customer">
│ Phone: [ 9800000000____ ]                     │          <label> + <input type="tel" name="phone">
│ Momo:  ( ) Steam  (•) Fried  ( ) Jhol  ← one  │          radios: same name="momo", different value
│ Extras: [x] Chutney  [ ] Soup        ← many   │          checkboxes
│ Plates: [ 2 ]                                 │          <input type="number">
│ Notes:  [ Less spicy please_____ ]            │          <textarea>
│                    [ Send to kitchen ]        │          <button type="submit">
└───────────────────────────────────────────────┘
The waiter (the server, later in the course) carries the slip to the kitchen.
The kitchen writes every answer in the column with that name:  customer=Aarav  phone=9800000000  momo=fried …
```

Words you now own

Term	What it means (plain English)	Example
`<form>`	The slip. It holds every field and the submit button	`<form method="get">`
`action`	Where the slip is sent. No `action` = the same page	`action="order.html"`
`method`	How it is sent. `get` puts the answers in the URL	`method="get"`
`<input>`	One box to fill. A void element. `type` decides what kind	`<input type="email">`
`<label>`	The printed word next to a box	`<label for="phone">Phone</label>`
`for` ↔ `id`	Ties a label to its box. Click the label → the cursor jumps into the box	`for="phone"` + `id="phone"`
`name`	The kitchen writes your answer in the column called `name`. **No name = not sent**	`name="phone"`
`value`	What is sent for a radio, checkbox or option	`value="fried"`
Radio group	"Choose one" circles: all share **one** `name`	`name="batch"`
Checkbox	"Tick any" boxes	extras
`<select>` / `<option>`	A drop-down list	momo type
`<textarea>`	A big box for many lines. Has a closing tag	notes
`<fieldset>` / `<legend>`	A box around a group of fields, with a title	"Choose your batch"
`required`	The browser won't send the form if this is empty	`<input required>`
`placeholder`	Grey example text inside the box. **Not** a label	`placeholder="98XXXXXXXX"`
Query string	The answers after `?` in the URL	`?customer=Aarav&momo=fried`

How it works

1. The URL trick: read what the form sends

With `method="get"` and no `action`, the form sends the slip to the same page and writes every answer into the URL bar. Submit, then read the address bar:

```
http://127.0.0.1:5500/index.html?fullname=Sunita+Gurung&email=sunita%40mail.com&batch=morning
                                 └──────┘ └──────────┘ └──────┘ └──────────┘
                                    name      value       name       value
     ?  starts the answers      &  separates them      +  is a space     %40  is @
```

If a field is **missing** from the URL, it has no `name` (D1-HW4 bug 10). If a radio shows `batch=on`, it has no `value` (D1-13 bug 4). This is the **D(URL)** tag.

2. Labels: always, and tied with for/id

- Every field needs a `<label>`. A placeholder disappears when you type, and screen readers may not read it.
- `<label for="X">` must match `id="X"` exactly. If they don't match, clicking the label does nothing (D1-13 bug 1, D1-15 bug 7, D1-HW1 bug 8). Test it: **click every label**.
- `name` and `id` often have the same word. They do different jobs: `id` ties the label; `name` is the column in the URL.

3. Pick the right `type`

type	Gives you	Use it for
`text`	a plain box	names
`email`	checks for an `@` on submit; email keyboard on phones	email (D1-HW4 bug 7)
`tel`	phone keyboard on mobile, accepts `+977`, spaces and dashes as typed	phone numbers (D1-13 bug 2, D1-HW1 bug 9)
`password`	hides the letters	passwords
`number`	spinner arrows; you cannot type `+` or a space, and the arrows drop a leading 0	a count of plates, an age
`date`	a date picker	trek date (D1-HW1 bug 10)
`radio`	choose one	batch, momo type
`checkbox`	tick any	extras
`color`	a colour picker	your TSI T-shirt colour

A phone number is **not** a number you do maths with (you never add two phone numbers). Use `tel`, not `number`. With `type="number"` you cannot type `+` or a space; the spinner arrow changes the number and drops a leading 0 (try 09841000000, press ↑). Use `tel`.

4. Radio groups: one name, different values

```
<input type="radio" name="batch" value="morning">   ← same name = one group = only one can be ticked
<input type="radio" name="batch" value="evening">
```

- Different names (`batch1`, `batch2`) make two separate groups, so both can be ticked (D1-13 bug 3, D1-HW1 bug 11, D1-HW4 bug 11).
- No `value` → the browser sends `on` (D1-13 bug 4).

5. Buttons

- `<button type="submit">` sends the slip. It must be **inside** `<form>…</form>`; after `</form>` it does nothing (D1-13 bug 6, D1-HW4 bug 12).
- `<button type="reset">` clears every field. Never label a reset button "Book now" (D1-HW1 bug 12).

6. `textarea` and `select` need closing tags

- `<textarea></textarea>` is **not** void. `<textarea />` does not close it, so the rest of the form becomes text inside the box (D1-13 bug 5, D1-15 bug 8). **Fix this first**: it hides other bugs.
- Every `<option>` lives inside `<select>…</select>`. An option after `</select>` shows as loose text (D1-HW4 bug 8).

7. Rules the browser checks for you

- `required` blocks an empty submit. A typo like `requred` is ignored, so empty forms go through (D1-HW4 bug 9).
- `type="email"` checks the `@.min`/`max` limit a number.
- This is only a first check for the visitor's convenience. A real server must check again (later in the course).

Code you built today

Step 1: form, label, text and email

File: `step1.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TSI course signup · Step 1</title>
</head>
<body>
  <h1>TSI course signup</h1>
  <form method="get">
    <p>
      <label for="fullname">Full name</label>
      <input type="text" id="fullname" name="fullname" placeholder="Sunita Gurung" required>
    </p>
    <p>
      <label for="email">Email</label>
      <input type="email" id="email" name="email" required>
    </p>
    <button type="submit">Sign up</button>
  </form>
</body>
</html>
```

Predict: type a name and `sunita@mail.com`, press Sign up. What appears after `?` in the URL bar? Then try an email with no `@`. What happens?

Step 2: radio and checkbox

File: `step2.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House order · Step 2</title>
</head>
<body>
  <h1>Momo House order slip</h1>
  <form method="get">
    <p>Choose your momo:</p>
    <p>
      <input type="radio" id="steam" name="momo" value="steam">
      <label for="steam">Steam</label>
      <input type="radio" id="fried" name="momo" value="fried">
      <label for="fried">Fried</label>
      <input type="radio" id="jhol" name="momo" value="jhol">
      <label for="jhol">Jhol</label>
    </p>
```

```
    <p>Extras:</p>
    <p>
      <input type="checkbox" id="chutney" name="extras" value="chutney">
      <label for="chutney">Extra chutney</label>
      <input type="checkbox" id="soup" name="extras" value="soup">
      <label for="soup">Soup</label>
    </p>
    <button type="submit">Send to kitchen</button>
  </form>
</body>
</html>
```

- Three radios, one `name`: only one can be ticked.
- **Predict:** tick both extras and submit. How many times does `extras=` appear in the URL?

Step 3: select and textarea

File: `step3.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House order · Step 3</title>
</head>
<body>
  <h1>Momo House order slip</h1>
  <form method="get">
    <p>
      <label for="filling">Filling</label>
      <select id="filling" name="filling">
        <option value="chicken">Chicken</option>
        <option value="veg">Veg</option>
        <option value="buff">Buff</option>
      </select>
    </p>
    <p>
      <label for="plates">Plates</label>
      <input type="number" id="plates" name="plates" min="1" max="10" value="1">
    </p>
    <p>
      <label for="notes">Notes for Didi</label>
      <textarea id="notes" name="notes" rows="3" cols="40"></textarea>
    </p>
    <button type="submit">Send to kitchen</button>
  </form>
</body>
</html>
```

- `<textarea …></textarea>`: open and close, nothing in between (unless you want starting text).
- **Predict:** write `Less spicy please` in Notes and submit. How are the spaces shown in the URL?

Step 4: fieldset, legend, submit, read the URL

File: `signup.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TSI course signup</title>
</head>
<body>
  <h1>TSI course signup</h1>
  <form method="get">
    <p>
      <label for="fullname">Full name</label>
      <input type="text" id="fullname" name="fullname" required>
    </p>
    <p>
      <label for="phone">Phone</label>
```

```
      <input type="tel" id="phone" name="phone" placeholder="98XXXXXXXX">
    </p>
    <fieldset>
      <legend>Choose your batch</legend>
      <input type="radio" id="morning" name="batch" value="morning">
      <label for="morning">Morning (Sun-Thu)</label>
      <input type="radio" id="evening" name="batch" value="evening">
      <label for="evening">Evening (Sun-Thu)</label>
    </fieldset>
    <p>
      <label for="start">Start date</label>
      <input type="date" id="start" name="start">
    </p>
    <p>
      <label for="shirt">Favourite colour for your TSI T-shirt</label>
      <input type="color" id="shirt" name="shirt" value="#990011">
    </p>
    <p>
      <label for="message">Why do you want to join?</label>
      <textarea id="message" name="message" rows="4"></textarea>
    </p>
    <button type="submit">Sign up</button>
    <button type="reset">Clear the form</button>
  </form>
</body>
</html>
```

- `fieldset` draws a box around the batch radios; `legend` is its title.
- The submit button is **inside** the form.
- **Predict:** pick the colour red and submit. The URL shows `shirt=%23990011`. What is `%23`? (It is `#`, written in a URL-safe way.)

Quick reference

You want...	Write
A form you can test	`<form method="get">` … `<button type="submit">Send</button></form>`
A labelled box	`<label for="x">X</label> <input type="text" id="x" name="x">`
Phone	`type="tel"`
Choose one	radios with the **same** `name`, different `value`
Tick many	checkboxes, each with a `value`
Drop-down	`<select name="…"><option value="…">…</option></select>`
Many lines	`<textarea name="…" rows="4"></textarea>`
A group with a title	`<fieldset><legend>…</legend>…</fieldset>`
Must not be empty	`required`

Mistakes everyone makes

You see...	Because...	Fix
Clicking the label does nothing	`for` and `id` don't match (D1-13 bug 1, D1-15 bug 7, D1-HW1 bug 8)	Same word in both
Spinner arrows on the phone box; `+977` is not accepted	`type="number"` (D1-13 bug 2, D1-HW1 bug 9)	`type="tel"`
Two batch circles can both be ticked	Different `name`s (D1-13 bug 3, D1-HW1 bug 11, D1-HW4 bug 11)	One shared `name`
URL shows `batch=on`	Radio has no `value` (D1-13 bug 4) — D(URL)	`value="morning"`
The rest of the form appears as text in a box	`<textarea />` (D1-13 bug 5, D1-15 bug 8)	`<textarea></textarea>` — fix this first
Submit does nothing	Button after `</form>` (D1-13 bug 6, D1-HW4 bug 12)	Move it inside
No date picker	`type="text"` for a date (D1-HW1 bug 10)	`type="date"`

You see...	Because...	Fix
"Book now" wipes the form	`type="reset"` (D1-HW1 bug 12)	`type="submit"`
Email with no `@` is accepted	`type="text"` (D1-HW4 bug 7)	`type="email"`
An option shows as loose text	`<option>` after `</select>` (D1-HW4 bug 8)	Move it inside
An empty form is sent	`requred` typo (D1-HW4 bug 9)	`required`
A field is missing from the URL	No `name` (D1-HW4 bug 10) — D(URL)	Add `name`
Clicking text next to a box does nothing	An input with no label (D1-FF4)	Add a `<label for>`

Check yourself

1. What are the jobs of `id` and `name`? Why do you often need both?
2. Why is `placeholder` not a replacement for `label`?
3. Which `type` for: a phone number, a trek date, a password, a count of plates?
4. How do you make three radio buttons "choose one"? What else must each one have?
5. **Predict:** what does the URL show for a checkbox with `name="extras"` and no `value`, when ticked?
6. What happens to a submit button placed after `</form>`?
7. Why should you fix a self-closed `<textarea />` first?
8. Explain the order-slip idea to a friend: what are the label, the name and the waiter?

Try it

- **D1-13 Broken Signup**: 6 bugs. Fix the textarea first; it hides bug 6. Submit and read the URL for bug 4.
- **D1-14 Extend the order slip**: a `tel` field, a Morning/Evening radio group with one name, a `fieldset` + `legend`. Bonus: `date`, two checkboxes, `type="color"`. Test by submitting and reading the URL.
- **D1-FF4 Evil page** (fast finishers): one bug is an input with no label.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. Example prompt:

> "After I submit my form, the URL shows `batch=on`. Explain what that means and which attribute controls it. Don't fix my code."

Remember the One Rule: you may only ship code you can explain, line by line. Next: 10 · Debugging and validating HTML.

10 · Debugging and Validating HTML: Student Notes

File: notes/10-debugging-and-validating-html.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2, Day 1 · Blocks B3 (spell-checker) + B9 + B10 · Layer: Frontend

Today in one sentence: The browser never tells you about your mistakes, so you need tools that do.

The picture to keep in your head

Browsers are **polite guessers** (they never show an error, they guess). HTMLHint = **spell-checker for HTML**.

- A polite guest at dinner never says "this food is too salty". They smile and eat. You never learn.
- The browser is that guest. Your HTML can be broken in ten places and it still smiles and shows *something*.
- A **spell-checker** underlines mistakes as you type. HTMLHint does that for HTML, inside VS Code.
- And in the capstone you are the **repair crew**: Didi's Momo House website broke the night before opening.

Words you now own

Term	What it means (plain English)	Example
Bug	A mistake in code that makes it do the wrong thing	`<titel>` instead of `<title>`
Debugging	Finding and fixing bugs, step by step	eyes → HTMLHint → F12
Silent bug	A bug you cannot see on the page	a missing `lang`
HTMLHint	A VS Code extension that underlines HTML mistakes	red squiggle under `<h1>`
`.htmlhintrc`	The file with the list of rules HTMLHint checks. The class one is in the `W2D1` folder	`"alt-require": true`
Problems panel	The list of every HTMLHint message (Ctrl+Shift+M)	"Tag must be paired"
Elements panel	F12 tab that shows the page the browser **built** (the DOM)	the boxes inside boxes
DOM	Document Object Model: the browser's own tree of boxes, built from your HTML	what Elements shows
View Source	The text you wrote, before the browser fixed anything	Ctrl+U
Validator	A website that checks your HTML against the official rules	validator.w3.org
Reveal	After every hunt, the instructor shows each bug and how you could see it	`solution/BUGS.md`

The bug tags (legend)

Every bug in class, homework and these notes carries one of these tags:

Tag	Meaning	Tool that shows it
V	You can **see** it on the page	your eyes
D(HH)	HTMLHint underlines it in VS Code / Problems panel (Ctrl+Shift+M)	HTMLHint with the class `.htmlhintrc`
D(F12)	Only the Elements panel shows what the browser actually built	Chrome DevTools
D(URL)	Submit the form (`method="get"`) and read the address bar	the URL bar
D(LH)	Lighthouse → Accessibility report (works offline)	Chrome DevTools, Day 2 only
D(W3C)	Only the W3C validator reports it	validator.w3.org (Wi-Fi bonus)
H	Hidden on Windows, breaks on a real Linux server (file-name case)	explained by the instructor
R	**Rule check**: no tool fires; you must read the code against the rule (e.g. "one h1", "no heading jumps")	your brain + the notes

How it works

1. The debugging routine (use it on every page)

```
1. EYES       Open with Live Server. Does it look right? Click every link. Click every label.   → V
2. HTMLHINT   Ctrl+Shift+M. Read each message, top to bottom. Fix the FIRST one first.          → D(HH)
3. F12        Elements panel: open the boxes. Does the tree match what you meant?               → D(F12)
              Network panel: any red 404 rows?                                                  → V
4. URL        Forms: submit with method="get" and read every name=value pair.                  → D(URL)
5. RULES      Read against the rules: one h1? no heading jumps? every img has alt?              → R
6. W3C        With Wi-Fi: validator.w3.org → "Validate by File Upload".                         → D(W3C)
```

Fix the first error first. One unclosed tag can cause five more messages below it. Fix one, save, look again.

2. HTMLHint: your spell-checker

The class `.htmlhintrc` switches on 12 rules:

Rule	What it catches	Class bug
`doctype-first`	No `<!DOCTYPE html>` on the first line	D1-04 bug 1, D1-HW4 bug 1
`html-lang-require`	`<html>` with no `lang`	D1-04 bug 2
`title-require`	No `<title>` in the head	D1-04 bug 3
`tag-pair`	An opening tag with no closing tag, or crossed tags	D1-04 bugs 4–5, D1-06 bugs 4–5, D1-FF5 bug 8
`alt-require`	`<img>` with no `alt`	D1-15 bug 2, D1-FF4, D1-HW4 bug 4
`id-unique`	The same `id` twice	D1-FF4
`tagname-lowercase`	`<P>` instead of `<p>`	—
`attr-lowercase`	`SRC=` instead of `src=`	—
`attr-value-double-quotes`	`href='x'` or `href=x`	—
`attr-no-duplication`	the same attribute twice on one tag	—
`src-not-empty`	`src=""`	—
`spec-char-escape`	a raw `<` or `>` in text	D1-06 bug 2 (sometimes)

Messages look like this in the Problems panel:

```
⚠ Tag must be paired, missing: [ </h1> ], start tag match failed [ <h1> ] on line 9.   htmlhint(tag-pair)   [Ln 9, Col 3]
⚠ An alt attribute must be present on <img> elements.                                   htmlhint(alt-require) [Ln 12, Col 3]
```

Click a message and VS Code jumps to that line.

What HTMLHint does NOT catch (so these are **R**): two `<h1>`s, heading jumps, `<b>` instead of `<strong>`, a heading used only for bold, a nav without links. Tomorrow, Lighthouse catches some of these.

3. View Source vs Elements: before and after the guess

- **View Source (Ctrl+U)** = exactly what you wrote.
- **Elements (F12)** = what the browser built after guessing and repairing.

When they are different, you have found a silent bug. The table below lists the repairs Chrome makes.

4. The browser repair table

Every row below was tested in Chrome. "You wrote" is in View Source. "Chrome built" is what the Elements panel shows.

Bug type	You wrote	Chrome built (Elements panel)	What you see	Class bug
Orphan `<td>` (no `<tr>`)	`<table><td>Steam</td><td>150</td></table>`	`<table><tbody><tr><td>Steam</td>…` — **an invented `<tr>`** (and `<tbody>`)	Looks fine	D1-FF5 bug 12, D1-HW4 bug 5

Bug type	You wrote	Chrome built (Elements panel)	What you see	Class bug
Text between rows	`</tr>Tea break 3:00<tr>`	The text is **moved above the table**	Text floats over the table	D1-11 bug 5
Misnested inline tags	`<p>I live in <strong>Kathmandu</p><p>Next</p></strong>`	`<p>I live in <strong>Kathmandu</strong></p><p><strong>Next</strong></p>` — **closed and re-opened** ("reconstructed")	The next paragraph turns bold too	D1-04 bug 5
Unclosed `<a>`, then another `<a>`	`<a href="a.html">Home<a href="b.html">Menu</a>`	`<a …>Home</a><a …>Menu</a>` — **closed by the next `<a>`**	Looks fine (silent)	D1-15 bug 3 (why it must be the last link)
Unclosed **last** `<a>`	`…<a href="b.html">Menu</nav><h1>Hi</h1>`	`<h1><a href="b.html">Hi</a></h1>` — the link is **re-opened** in every following block	The `<h1>` and more become a link	D1-15 bug 3
A second `<head>`	`<title>T</title><head><meta …></head>`	**The second `<head>` tag is ignored**; its `<meta>` joins the first head	Looks fine	D1-FF5 bug 1
Content after `</html>`	`</body></html><p>Contact Didi</p>`	`<p>Contact Didi</p>` **moved into `<body>`**	Looks fine	D1-FF5 bug 2
Unclosed `<title>`	`<title>Kathmandu Treks</head><body><h1>…`	The **whole rest of the file becomes the title text**; `<body>` is empty	A blank page	D1-HW1 bug 1
`<textarea />`	`<textarea name="n" /><button>Send</button></form>`	`/>` is ignored; **the rest of the file becomes text inside the box**	Code shows inside the box; the button is gone	D1-13 bug 5, D1-15 bug 8
Unclosed comment `->`	`<!-- note -> <footer>…</footer></body></html>`	**Everything after it is inside the comment**	The footer and the rest disappear	D1-04 bug 6
Unknown tag `<titel>` in the head	`<head><titel>Aarav's Page</titel></head>`	`<titel>` is **moved into `<body>`**; the page has no title	The tab shows the file path; the words show on the page	D1-04 bug 3
`<title>` inside `<body>`	`<body><title>Inside body</title>`	It stays in the body but is hidden; **the tab still uses it**	Looks fine (that is the lesson)	D1-HW4 bug 2
`<ul>` inside `<p>`	`<p>Intro<ul><li>a</li></ul></p>`	`<p>Intro</p><ul>…</ul><p></p>` — **the `p` closes early**, plus an empty `p`	Looks almost fine	D1-FF4
`<h1>` inside `<p>`	`<p>One<h1>Big</h1></p>`	`<p>One</p><h1>Big</h1><p></p>` — **the `p` closes early**	Looks almost fine	D1-FF4
`<a>` inside `<a>`	`<a href="x.html">Out<a href="y.html">In</a></a>`	Two links side by side: **the outer `<a>` is closed**	Two links, not one	D1-FF4
`</hr>`	`<p>a</p></hr><p>b</p>`	**Ignored** (no `<hr>` is made)	The divider is missing	D1-FF5 bug 6
`</quote>` for a blockquote	`<blockquote>q</quote><p>after</p>`	`</quote>` ignored; **the blockquote stays open** to the end	The rest of the page is indented	D1-06 bug 4
Entity typo `&nbps;`	`Rs.&nbps;150`	Printed **letter by letter**	`Rs.&nbps;150` on the page	D1-06 bug 3
No `<!DOCTYPE html>`	`<html><head>…`	The page runs in **quirks mode** (`document.compatMode` = `BackCompat`)	Small layout differences; CSS on Day 2 behaves oddly	D1-04 bug 1
What Chrome does **not** repair	`<li>` with no `<ul>`; `<li>` in a `<div>`; a `<ul>` placed after `</li>`; text directly in `<ul>`	**Left exactly as written** (invalid, but kept)	The text still shows, so it looks almost right (a lone `<li>` loses its indent and its bullet)	D1-07 bug 1, D1-FF5 bugs 7 and 9, D1-HW1 bug 4

The big lesson: **"it looks fine" does not mean "it is fine".** Google, screen readers and your future JavaScript read the built tree, not your intentions.

5. The W3C validator (Wi-Fi bonus)

1. Go to **validator.w3.org**.
2. Choose **Validate by File Upload** and pick your `index.html` (or paste it under **Validate by Direct Input**).
3. Read each **Error** from the top. Fix the first, save, upload again.

It catches some things HTMLHint cannot: a `<caption>` in the wrong place (D1-HW4 bug 6), content after `</html>`, a `<header>` inside `<footer>` (Day 2).

6. The capstone relay rules (D1-15)

- Pairs. One **driver** types; one **navigator** reads the README and points. Swap after every bug.
- 10 bugs across 4 pages (3 / 3 / 2 / 2).
- **1 point per bug + 1 bonus point** if you name its tag (V, D(HH), D(F12), R…).
- On `order.html`, fix the `<textarea />` first.
- Finished? Become **helpers** for another pair: you may point, not type. Then move on to the fast finishers.

Code you built today

Step 1: before (the B10 demo page, 5 errors)

File: `before.html`

✗ Broken on purpose: find what is wrong

```
<html>
<head>
  <meta charset="UTF-8">
  <titel>Kathmandu Gallery</titel>
</head>
<body>
  <h1>Kathmandu Gallery
  <img src="images/swayambhu.svg" width="400">
  <p>Visit <a href="https://en.wikipedia.org/wiki/Swayambhunath">Swayambhu</a>.</p>
  <img src="images/boudha.svg" alt="Boudha stupa with prayer flags" width="400">
</body>
</html>
```

Predict: how many messages will the Problems panel show? Write your guess, then press Ctrl+Shift+M. (The errors: no DOCTYPE, no `lang`, `<titel>` so no title, an unclosed `<h1>`, an `<img>` with no `alt`.)

Step 2: after (zero problems)

File: `after.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Kathmandu Gallery</title>
</head>
<body>
  <h1>Kathmandu Gallery</h1>
  <img src="images/swayambhu.svg" alt="Swayambhu stupa on its hill above Kathmandu" width="400">
  <p>Visit <a href="https://en.wikipedia.org/wiki/Swayambhunath">Swayambhu</a>.</p>
  <img src="images/boudha.svg" alt="Boudha stupa with prayer flags" width="400">
</body>
</html>
```

Step 3: see a repair with your own eyes

Type this broken table, open it, and compare **View Source** with **F12 → Elements**.

File: `repair.html`

✗ Broken on purpose: find what is wrong

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Repair test</title>
</head>
<body>
  <h1>Repair test</h1>
  <table>
    <tr><td>Steam</td><td>Rs. 150</td></tr>
    Tea break 3:00
    <tr><td>Fried</td><td>Rs. 180</td></tr>
  </table>
  <p>I live in <strong>Kathmandu</p>
  <p>Where is the bold?</p>
</body>
</html>
```

Predict: where will "Tea break 3:00" appear? Will "Where is the bold?" be bold? Then check in Elements: find the `<strong>` that Chrome added on its own.

Step 4: the self-audit checklist (D1-16)

Run this on your own D1-10 gallery folder. Tick each line only when it is true.

```
[ ] Line 1 is <!DOCTYPE html>
[ ] <html lang="en">
[ ] <meta charset="UTF-8"> and the viewport meta are in the head
[ ] A real <title> (not "Document")
[ ] Exactly one <h1>; no heading jumps
[ ] Every <img> has a real alt (or alt="" for decoration)
[ ] Every link works: click each one; external links start with https://
[ ] File and folder names are lowercase, no spaces
[ ] The Problems panel (Ctrl+Shift+M) says "No problems"
[ ] Bonus with Wi-Fi: the W3C validator shows no errors
```

Quick reference

You want to...	Do this
See HTMLHint messages	Ctrl+Shift+M
See what the browser built	F12 → Elements (click the ▶ arrows)
See exactly what you wrote	Ctrl+U
Find a broken path	F12 → Network → F5 → red 404 rows
Check a form	Submit with `method="get"`, read the URL
Official check	validator.w3.org → Validate by File Upload
Fix messy nesting	Shift+Alt+F, then look at the indentation

Mistakes everyone makes

You see...	Because...	Fix
"It looks fine, so it's done"	The browser repaired it silently (the repair table above)	Always do steps 2 and 3 of the routine
20 messages and you panic	One unclosed tag causes many messages below it (D1-04 bug 4)	Fix the first message, save, look again
No squiggles at all	HTMLHint not installed, or you opened a file, not the `W2D1` folder	Install it; File → Open Folder
You fix the page by deleting the broken part	That removes the content Didi needs	Repair, don't delete
Blank page, nothing in Elements	An unclosed `<title>` swallowed the page (D1-HW1 bug 1)	Close `</title>` first

You see...	Because...	Fix
The form looks short and code shows in a box	`<textarea />` swallowed the rest (D1-13 bug 5, D1-15 bug 8)	Fix the textarea first
HTMLHint is quiet but a heading is wrong	Heading rules are **R**: no tool checks them (D1-06 bug 1, D1-15 bug 6)	Read against the rule

Check yourself

1. Why do we call browsers "polite guessers"? Give one example of a guess.
2. What is the difference between View Source and the Elements panel?
3. Name three things HTMLHint catches and two it does not.
4. What does the browser do with text written between two `<tr>`s?
5. What does the browser do with an unclosed `<a>` when the next tag is another `<a>`? And when it is the last link?
6. Why should you fix the first HTMLHint message first?
7. Which tag would you give "two `<h1>` on one page"? Why that tag?
8. Explain to a friend why "it looks fine" is not enough, in three sentences.

Try it

- **D1-04 Broken Skeleton**: the first hunt with HTMLHint + F12.
- **D1-15 Momo House repair** (pairs): 10 bugs across 4 pages. Name the tag for a bonus point.
- **D1-16 Validate & self-audit**: the checklist above on your own D1-10 folder.
- **D1-FF4 Evil page**: 8 "the browser hides it" bugs. Use the repair table.
- **D1-FF5 Notice Board**: 12 bugs of every kind.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. Debug with AI only **after** you tried the routine yourself. Example prompt:

> "In View Source my page has `<p>I live in <strong>Kathmandu</p>`, but the Elements panel shows an extra `<strong>` in the next paragraph. Why did Chrome add it? Explain the rule, don't fix my file."

Remember the One Rule: you may only ship code you can explain, line by line. Next: 11 · Day 1 cheatsheet.

11 · Day 1 Cheatsheet: How the Web Works + HTML Core

File: notes/11-day1-cheatsheet.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2, Day 1 · Layer: Frontend

Keep this page open while you code. Details and examples are in notes 01–10.

The analogies

Topic	Remember it as
The Internet	**Postal system**: request = letter, DNS = phone book, IP = home address, server = warehouse, response = parcel
VS Code + Live Server	**Professional kitchen**; Live Server = "taste as you cook"; Emmet = **speed-dial for tags**
Skeleton	**Labeled boxes inside boxes**. Last box opened, first box closed
head / body	**The label on the parcel** / **what's inside the parcel**
Text tags	**Tone of voice**
Entities	**Escape codes**: `<` = "print a <, don't start a tag"
Lists	**Shopping list** (ul) vs **recipe steps** (ol)
Paths	**Same house vs postal address**; `../` = "go downstairs one floor"
Images	**A sticker, not an envelope**; alt = the description you read over the phone
Tables	**Four tags, one team**; the canteen price board
Forms	**The order slip**; name = the column the kitchen files your answer under
Debugging	Browsers are **polite guessers**; HTMLHint = **spell-checker for HTML**

The skeleton

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House · Home</title>
</head>
<body>
  <h1>Momo House</h1>
  <p>Fresh momo every day.</p>
</body>
</html>
```

Tags on one page

Group	Tags
Text	`h1`–`h6` (one h1, no jumps) · `p` · `strong` · `em` · `br` · `hr` · `blockquote` · `q` · `span`
Entities	`<` `>` `&` `©` ` ` `→` `♥` `"`
Lists	`ul` / `ol` (`type`, `start`, `reversed`) → `li` · `dl` → `dt` + `dd` · nested list **inside** the `li`
Links	`<a href="…">` · `https://` for other sites · `pages/x.html` · `../x.html` · `#id` · `mailto:` · `tel:` · `target="_blank"`
Images	`<img src="…" alt="…" width="300">` (void) · `alt=""` for decoration · `figure` + `figcaption`
Tables	`table` · `caption` (first) · `thead` / `tbody` / `tfoot` · `tr` · `th scope="col"` or `"row"` · `td` · `colspan` · `rowspan`
Forms	`form method="get"` · `label for` ↔ `input id` · `name` · `value` · `select` / `option` · `textarea` (closing tag!) · `fieldset` / `legend` · `required` · `placeholder` · `button type="submit"` or `"reset"`
Input types	`text` `email` `tel` `password` `number` `date` `radio` `checkbox` `color`
Void elements	`meta` `link` `img` `br` `hr` `input` — no closing tag

Emmet

Type	Get
`!`	full skeleton (then change the title!)
`h1+p`	h1 next to p
`ul>li*5`	list with 5 items
`nav>a*3`	nav with 3 links
`table>tr*3>td*2`	3 rows × 2 cells
`.card>h2+p`	div.card with h2 and p
`ul>li.item$*3`	item1, item2, item3
`lorem20`	20 dummy words

Shortcuts

Ctrl+S save · Alt+Shift+↓ copy line · Alt+↑/↓ move line · Ctrl+/ comment · Shift+Alt+F format · Ctrl+D next match · Ctrl+Shift+M Problems · Ctrl+\` terminal · F12 DevTools · Ctrl+U View Source

Paths in one picture

```
site/
├── index.html          → pages/about.html    · images/momo.svg
├── images/momo.svg
└── pages/about.html    → ../index.html       · ../images/momo.svg    · menu.html (same floor)
```

Lowercase · no spaces · exact extension (`.html`, not `.htm`) · never `C:\…` · external = `https://…`

Counting table cells

Every row adds up to the same number of columns. `colspan="3"` counts 3. A `rowspan="2"` cell takes a seat in the next row too, so that row writes one cell less.

Forms: the URL trick

`<form method="get">` → submit → `?name=value&name=value`. `on` = a radio/checkbox with no `value`. A field missing from the URL = no `name`.

The debugging routine

1. **Eyes** (V) · 2. **HTMLHint** Ctrl+Shift+M (D(HH)) · 3. **F12 Elements + Network** (D(F12)) ·
4. **Submit and read the URL** (D(URL)) · 5. **Read against the rules** (R) · 6. **W3C** with Wi-Fi (D(W3C)).

Fix the first error first. **Fix these first:** an unclosed `<title>` (blank page), a `<textarea />`, an unclosed comment.

Bug tags

V see it · **D(HH)** HTMLHint · **D(F12)** Elements panel · **D(URL)** address bar · **D(LH)** Lighthouse (Day 2) · **D(W3C)** validator · **H** case on Linux · **R** rule check (no tool).

Top 12 Day 1 bugs

Bug	Fix	Tag
No DOCTYPE	`<!DOCTYPE html>` line 1	D(HH)
No `lang`	`<html lang="en">`	D(HH)
`<titel>` / unclosed `<title>`	`<title>…</title>`	V + D(HH)

Bug	Fix	Tag
Crossed tags `<p><strong></p></strong>`	Last opened, first closed	D(HH)
Comment ends `->`	`-->`	V
Two `<h1>`, heading jumps	One h1, next level only	R
`&nbps;`	` `	V
`<li>` with no `<ul>`	Wrap in `<ul>`	V + D(F12)
`image/`, `about.html` from the wrong floor, no `https://`	Check folder, floor, scheme	V
`<img>` with no alt	A real description	D(HH)
Row with too few/many cells, `colspam`	Count every row	V
`for`/`id` mismatch, `<textarea />`, button after `</form>`	Match, close, move inside	V

AI rule (Stage 1: AI as Tutor)

AI may **explain** a concept or an error. You type every line. Try first, then ask: "Explain what this HTMLHint message means. Don't rewrite my code."

Remember the One Rule: you may only ship code you can explain, line by line. Next session (Day 2): Semantic HTML, accessibility and media, then CSS basics. The skeleton gets its outfit.